BrushQuadX Drone

Release v3.0

JohnMVSantos

Oct 29, 2026

CONTENTS:

1 Introduction **2**

2 Drone Tutorial **3**
- 2.1 Materials . . . 4
- 2.2 Hardware . . . 5
- 2.3 Software . . . 15

3 Controller Tutorial **17**
- 3.1 Materials . . . 18
- 3.2 Hardware . . . 19
- 3.3 Software . . . 25

4 Concepts **27**
- 4.1 Metal Oxide Semiconductor Field-Effect Transistor (MOSFET) . . . 28
- 4.2 Motors and Lenz' Law . . . 30
- 4.3 Uni-directional Motor Driver . . . 33
- 4.4 Bi-directional Motor Driver . . . 34
- 4.5 PWM Signals . . . 36

5 Reflection **37**
- 5.1 Protoype 1 - June 8, 2025 . . . 38
- 5.2 Protoype 1.1 - June 22, 2025 . . . 41
- 5.3 Prototype 1.2 - August 18, 2025 . . . 43
- 5.4 Prototype 1.3 - August 31, 2025 . . . 45
- 5.5 Prototype 1.4 - November 30, 2025 . . . 46
- 5.6 Prototype 1.5 - January 31, 2026 . . . 48
- 5.7 Prototype 2.0 - March 28, 2026 . . . 50
- 5.8 Prototype 2.1 - April 26, 2026 . . . 53
- 5.9 Prototype 3.0 - September 30, 2026 . . . 56
- 5.10 Prototype 3.0 - October 30, 2026 . . . 58
- 5.11 Key Learnings - October 08, 2026 . . . 59

6 Future Work **60**

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

INTRODUCTION

This is the official documentation for the BrushQuadX project. The BrushQuadX is a quadcopter drone that uses brushed motors and is designed for educational and hobbyist purposes. This documentation provides an overview of the project, tutorials for getting started, and detailed explanations of the concepts and components involved in building and operating the BrushQuadX.

Explore the sections of the documentation below:

1. **Drone Tutorial** Learn more about the step-by-step tutorial for building the drone. See the full guide in the *Drone Tutorial* section.
2. **Controller Tutorial** Learn more about the step-by-step tutorial for building the controller. Check out the complete guide in the *Controller Tutorial* section.
3. **Concepts** Understand the fundamental concepts and principles behind the BrushQuadX design. See the full details in the *Concepts* section.
4. **Prototypes** Learn more about the various prototypes and the evolution of the design throughout the project. See the full details in the *Prototypes* section.
5. **Future** See some of the future work planned for the BrushQuadX project. See the full details in the *Future* section.

CHAPTER

TWO

DRONE TUTORIAL

This page will show the step-by-step tutorial for building the BrushQuadX drone. The tutorial will cover the hardware assembly, software installation, and configuration required to get the drone up and running.

2.1 Materials

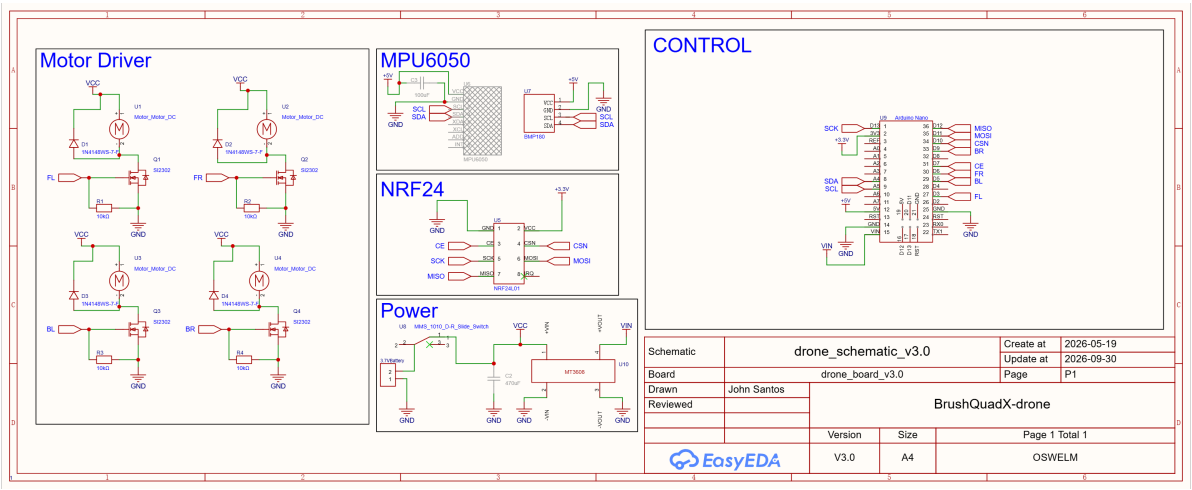
1. Arduino Nano Atmega328P (x1)
2. MPU6050 Gyroscope/Accelerometer (x1)
3. BMP280 Barometric Atmospheric Altitude Pressure Sensor (x1)
4. NRF24L01 (x1)
5. Micro 600TVL FPV Camera with 5.8GHz 25mW Transmitter (x1)
6. Si2302 n-channel MOSFET (x4)
7. 1N4148 Diode (x4)
8. 0805 10k Ohm Resistor (x4)
9. Brushless Motor (x4)
10. 100uF, 470uF capacitor (x1)
11. MMS1010 D-R slide switch (x1)
12. 3.7V 30C 450mAH LiPO (x1)
13. MT3608 Boost Converter (x1)
14. 46mm 0.8mm propeller (x4)
15. Molex 51005 Female Battery Connector (x1)
16. M2 2cm nylon hex standoffs (x4)

2.1.1 Common

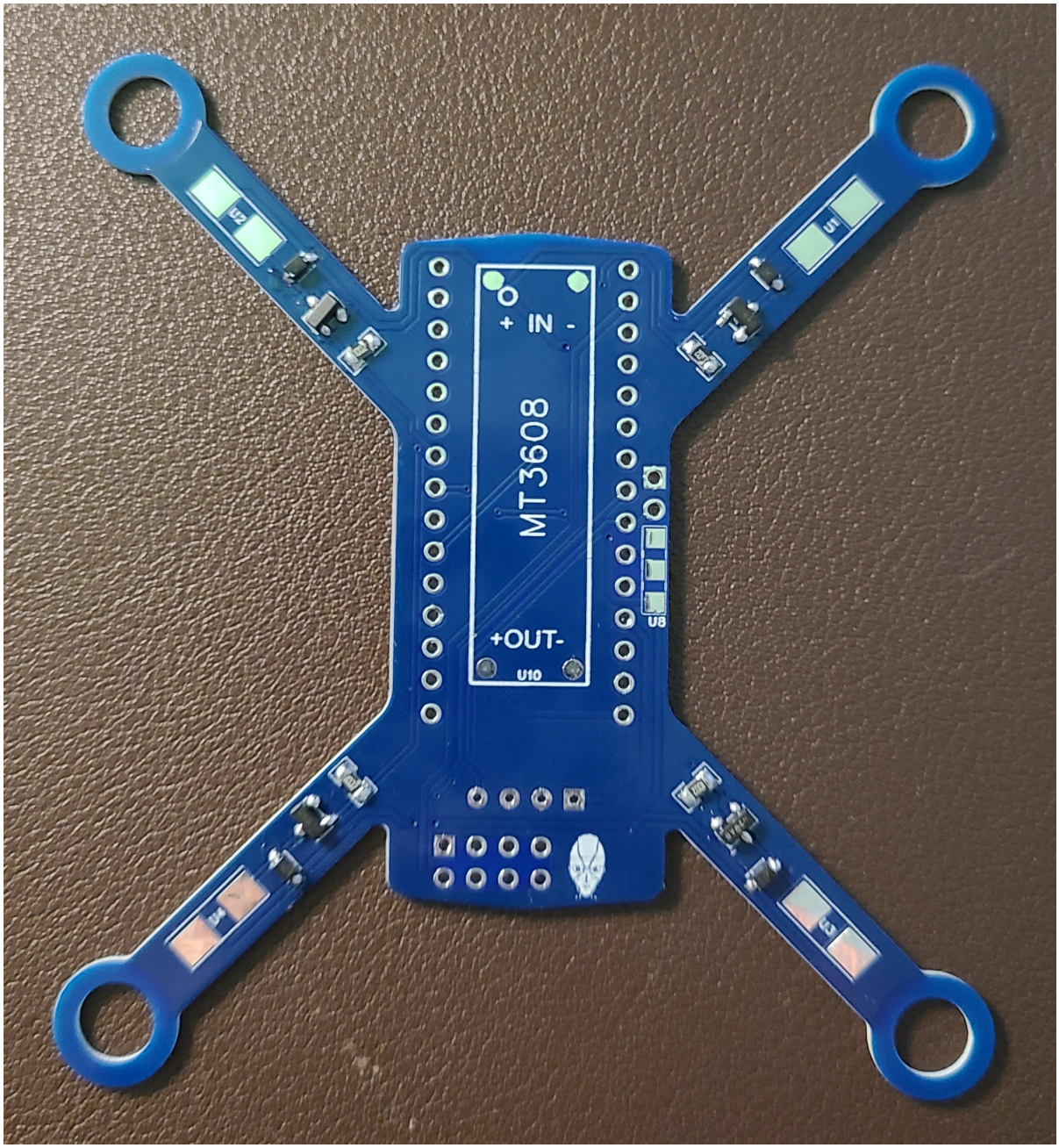
1. Super Glue
2. Soldering Iron and Solder
3. Mini USB to TTL Serial Converter Adaptor
4. M2 Nylon Hex Spacer Standoff Kit with Male and Female Screw Nut, etc.
5. 30AWG wires

2.2 Hardware

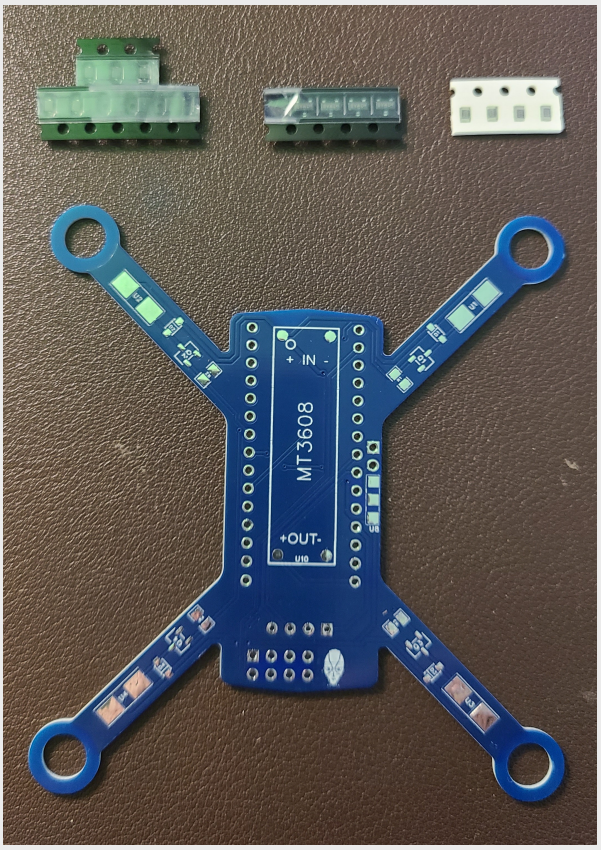
1. Take the Gerber files for the drone PCB and upload them to JLCPCB to manufacture the PCB. The Gerber file can be found in the *design_files/Gerber_Drone_PCB_v3.0_2026-10-02.zip*.
2. The drone PCB is based on the following schematic.

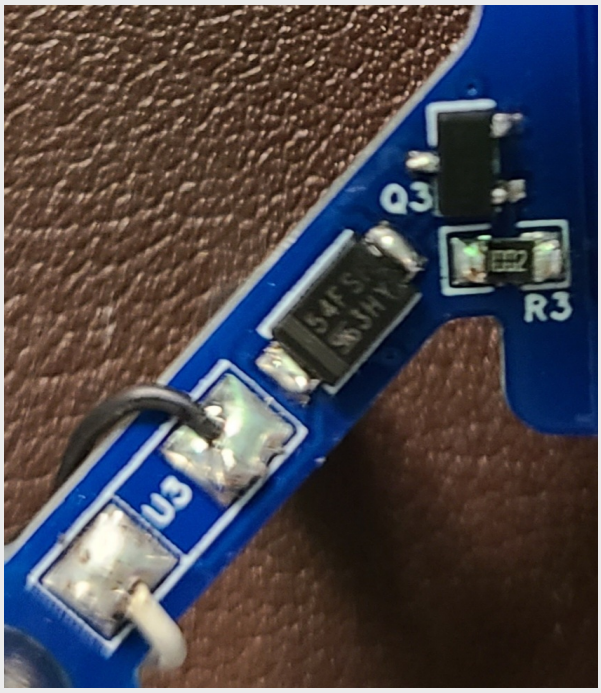


3. The PCB should appear as the following when it is received.

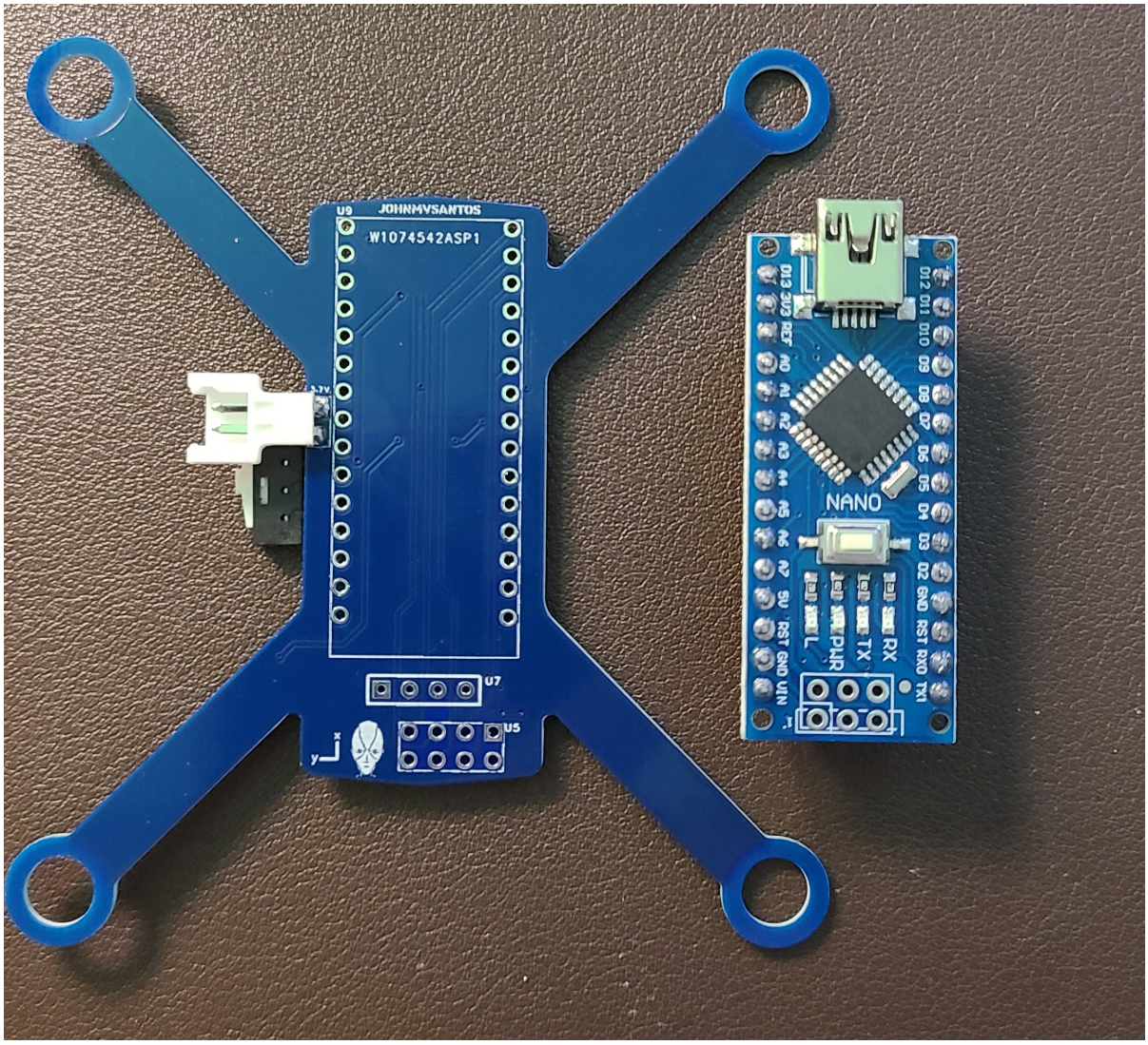


4. Assemble and solder the motor driver of the drone along the arms with 10KOhm resistor 0805, 1N4148 surface mount diodes, and Si2302DS n-channel MOSFETS.

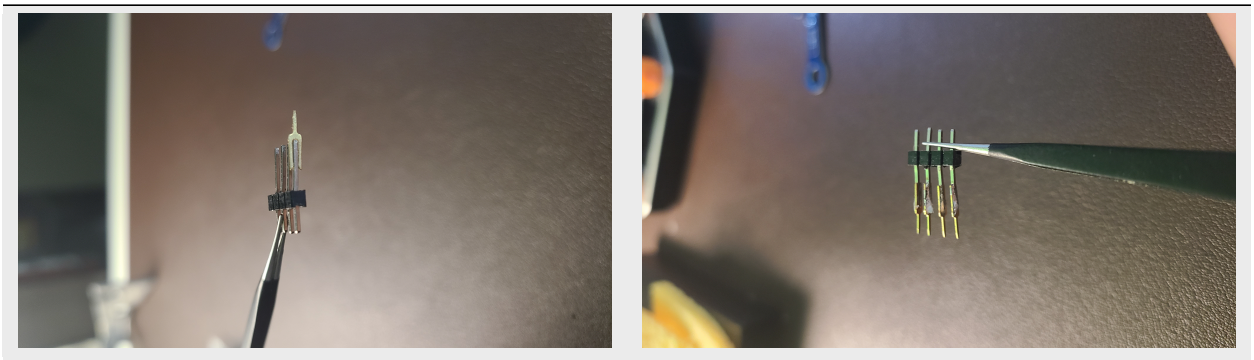




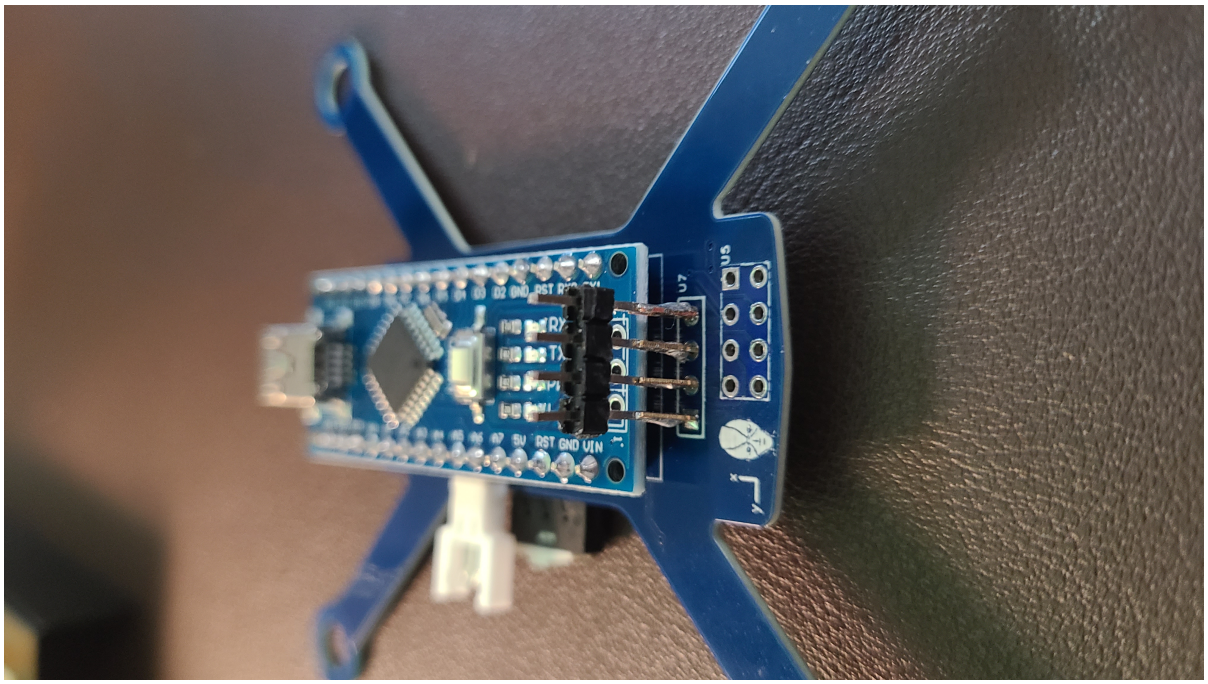
5. Solder the MMS1010 D-R slide switch and the Molex 51005 Female Battery Connector to the PCB as shown. Then solder the Arduino Nano to the PCB.



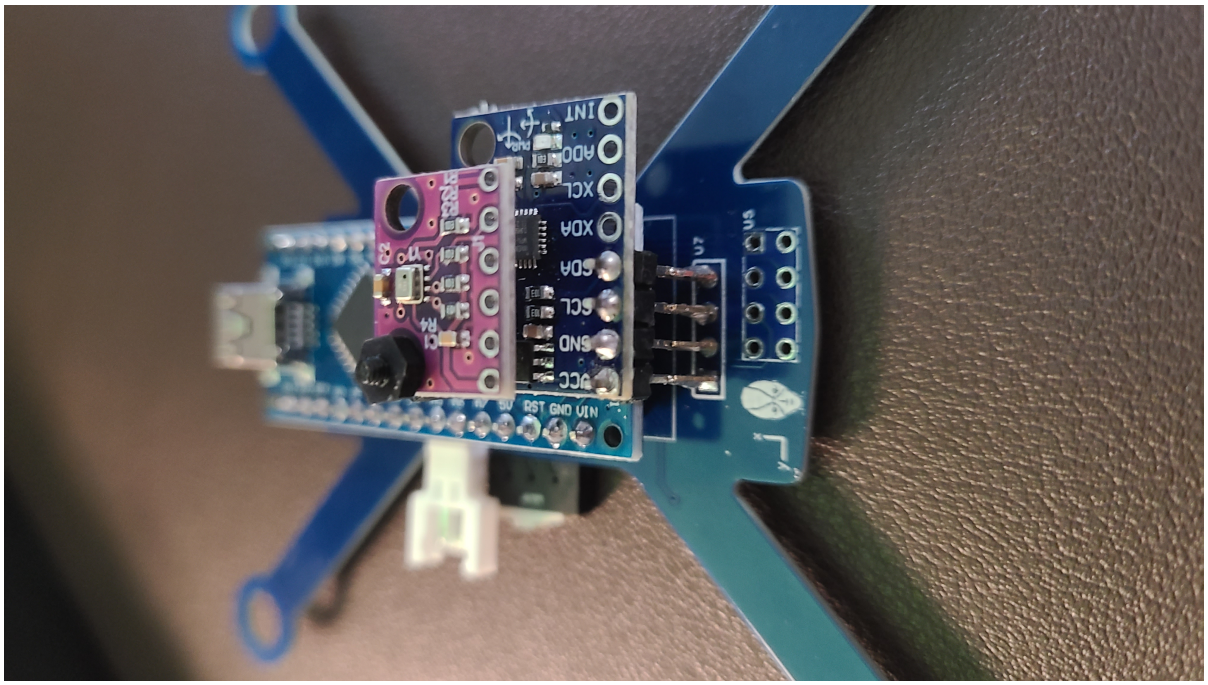
6. Create an extended 4 pin male to male adaptor as shown in the following image to connect the MPU6050 and BMP280 to the PCB.



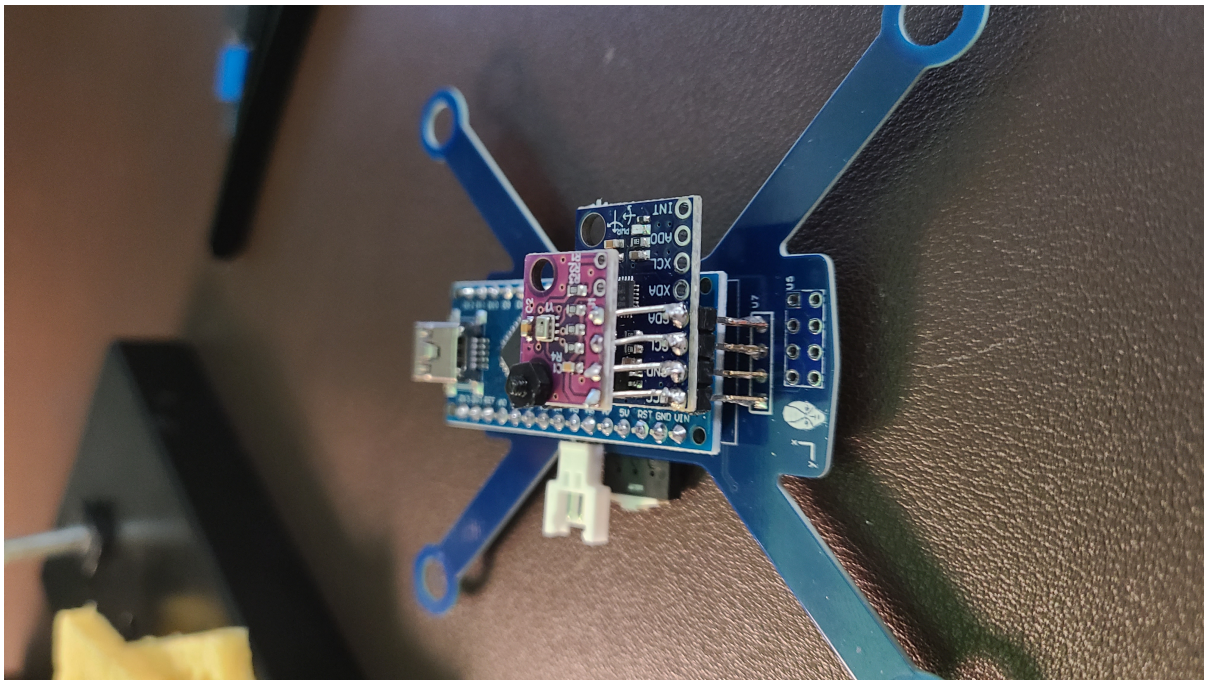
7. Solder the extended pin connection to the PCB.



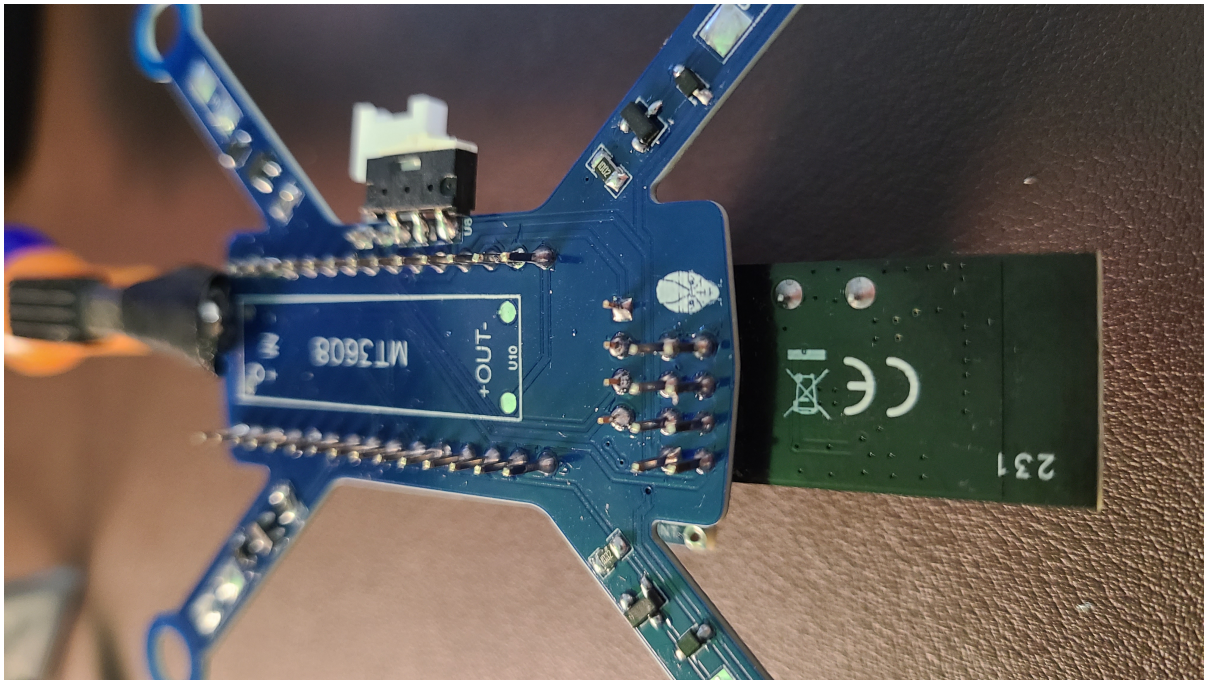
8. Solder the MPU6050 to the extended pin connection. Also connect a 100uF capacitor to the VCC and GND of the MPU6050 to reduce noise.



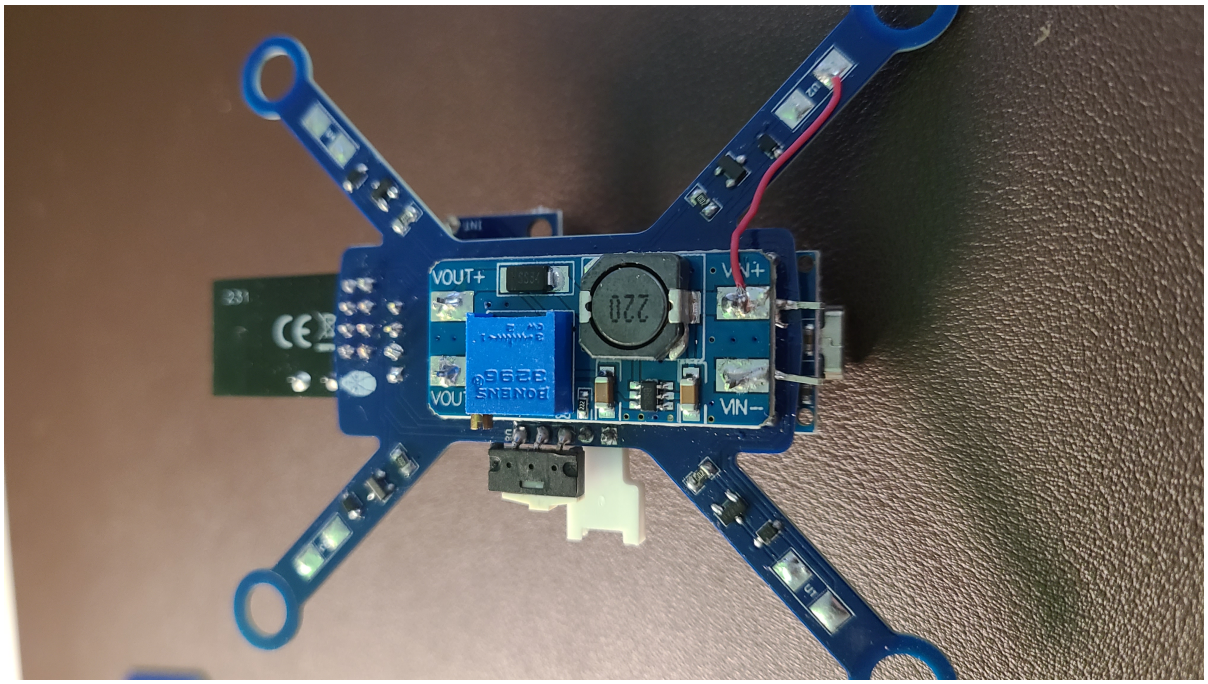
9. Solder the BMP280 to the MPU6050.



10. Solder the NRF24L01 module to the PCB.



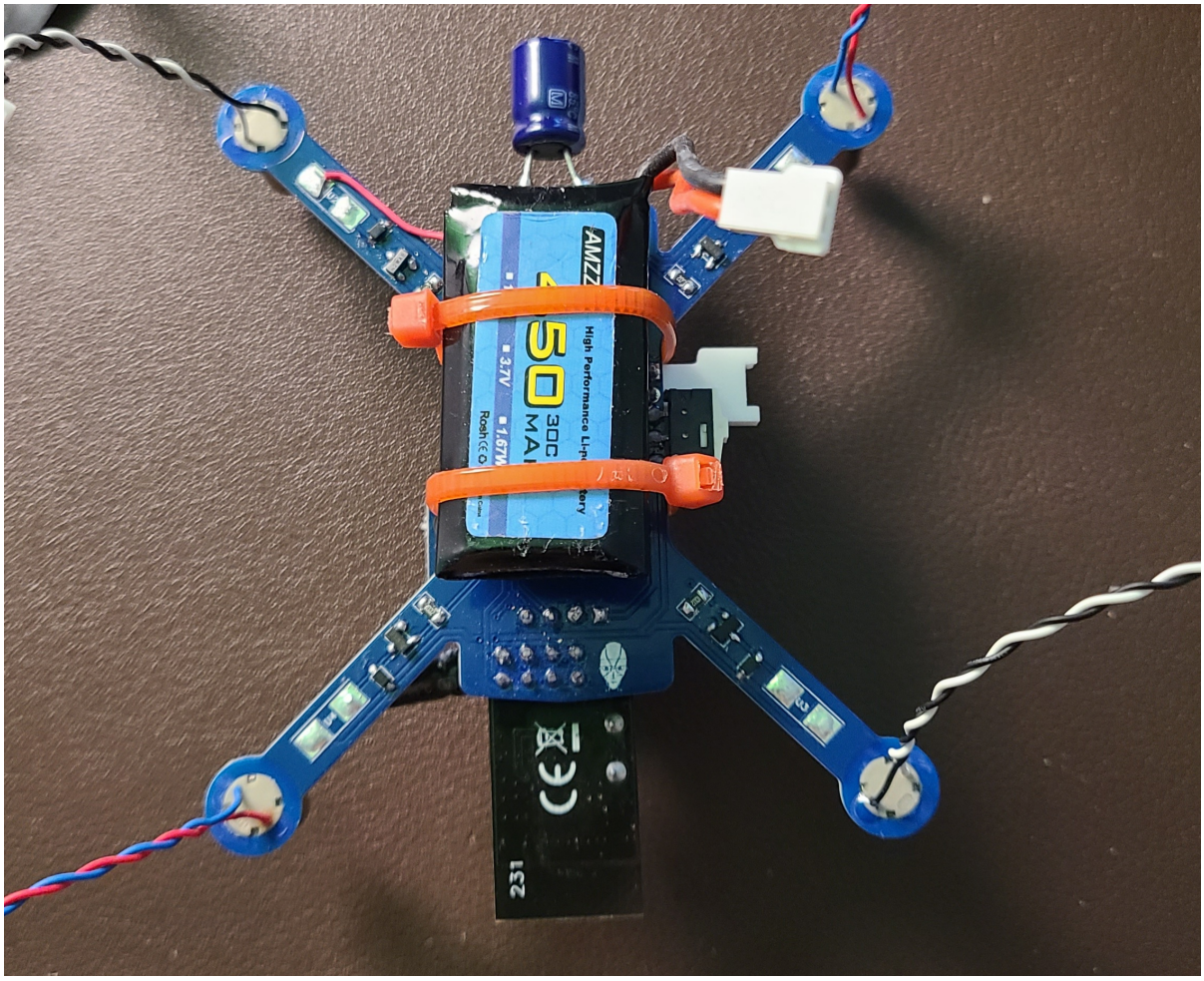
11. Solder the MT3608 DC-DC Boost Converter to the PCB. Also connect a 470uF capacitor to the VCC and GND of the MT3608 to reduce noise.



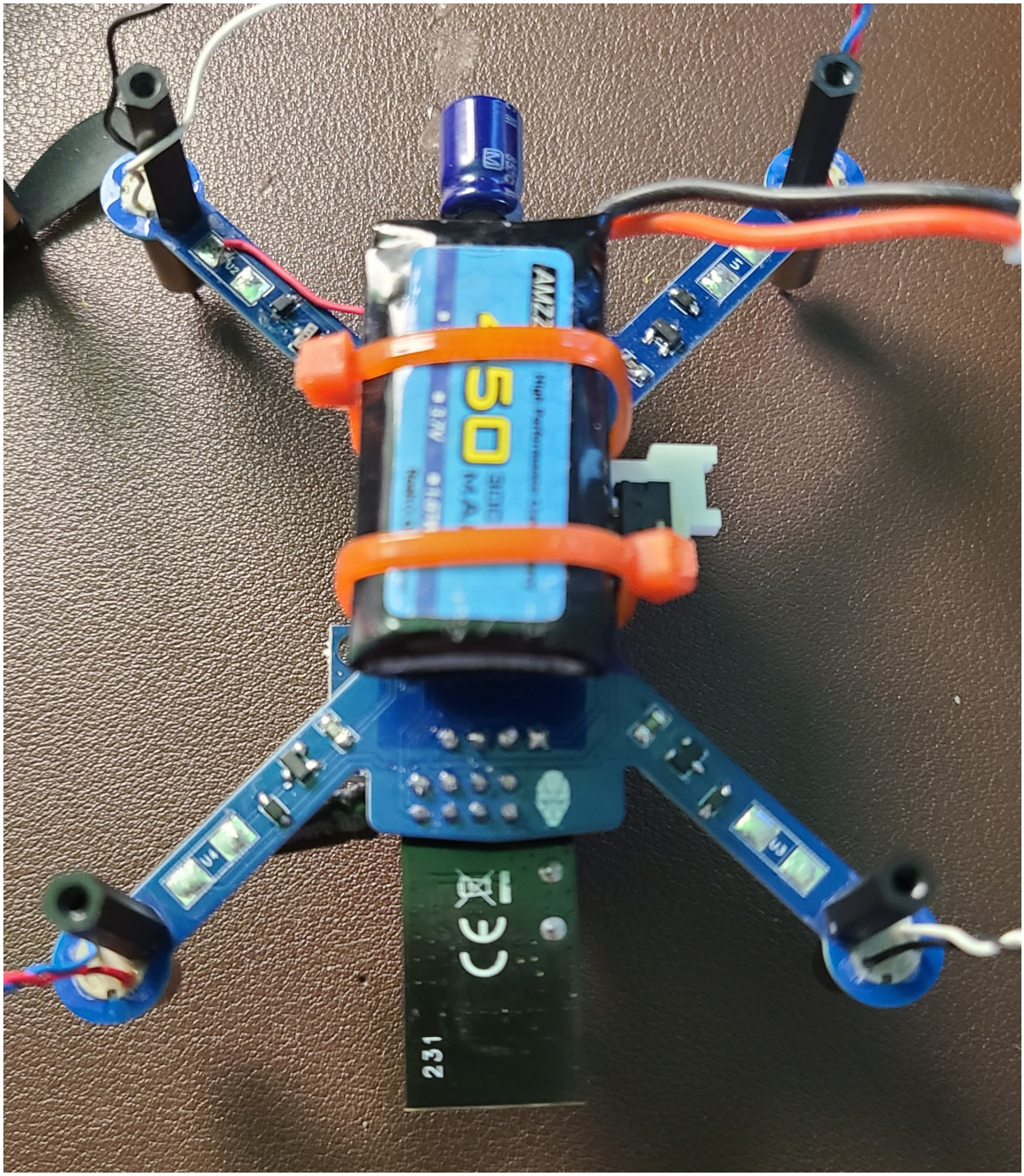
12. Superglue the brush motors to the through holes of the PCB based on the following orientation:
    - Top Left: Motor 1 (CW Red/Blue)
    - Top Right: Motor 2 (CCW White/Black)
    - Bottom Left: Motor 3 (CCW White/Black)
    - Bottom Right: Motor 4 (CW Red/Blue)

The wires are soldered to the PCB by having the red or white power lines connected to the positive terminal which is closest to the motor.

Then attached the battery to the drone using zip ties.



13. Attach M2 2cm nylon hex standoffs to the bottom of the PCB as lefts for the drone as shown.



14. The FPV camera can be powered by connecting the red and black wire to the VCC and GND input pins of the MT3608 DC-DC Boost Converter. The camera can be attached on top of Arduino Nano USB port using super glue. In the following image, a DIY platform was used to hold the camera using zip ties.

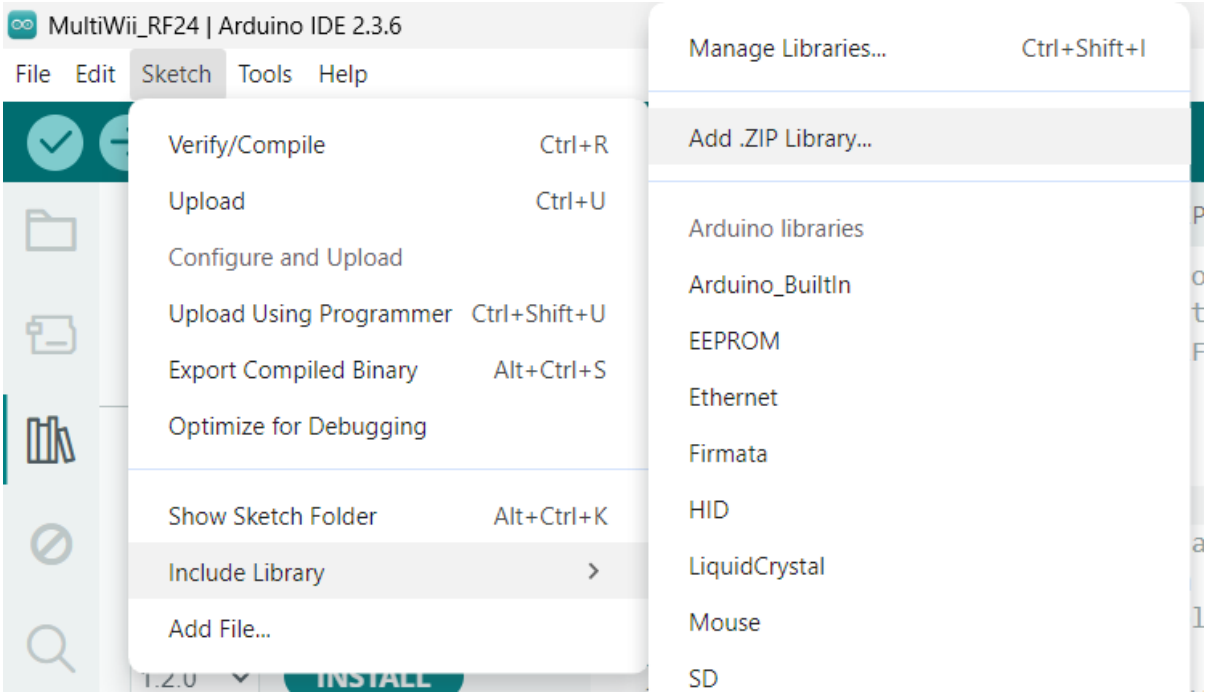
15. The final assembled drone should look like the following.

9V
6LR61
Ultra alkaline
PKCELL
Green energy, easy life

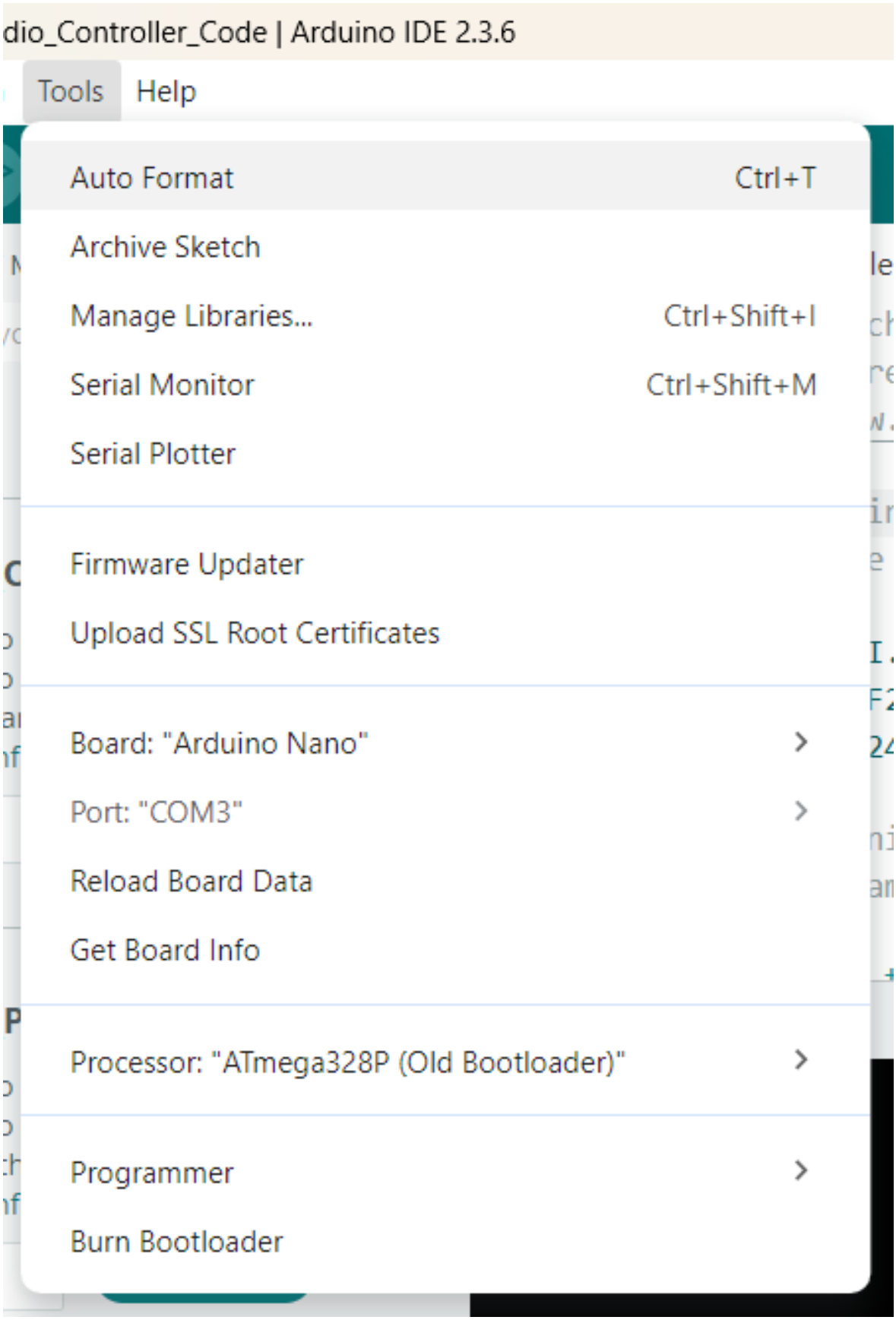
2.3 Software

1. Clone the https://github.com/BrushQuadX/brushquadx-rf24-rxdrone repository.
2. Open Arduino IDE and open the project *BrushQuadX_RF24_RXDrone*.
3. Install the following libraries and include the ZIP libraries in the Arduino IDE.

- RF24
- TimerFreeTone_v1.5



4. Adjust the upload settings in Arduino under Tools to set the right board “Arduino Nano”, the COM Port, and the processor to “ATmega328P (Old Bootloader)”.


dio_Controller_Code | Arduino IDE 2.3.6
Tools
Help
Auto Format
Ctrl+T
Archive Sketch
Manage Libraries...
Ctrl+Shift+I
Serial Monitor
Ctrl+Shift+M
Serial Plotter
Firmware Updater
Upload SSL Root Certificates
Board: "Arduino Nano"
Port: "COM3"
Reload Board Data
Get Board Info
Processor: "ATmega328P (Old Bootloader)"
Programmer
Burn Bootloader

CHAPTER

THREE

CONTROLLER TUTORIAL

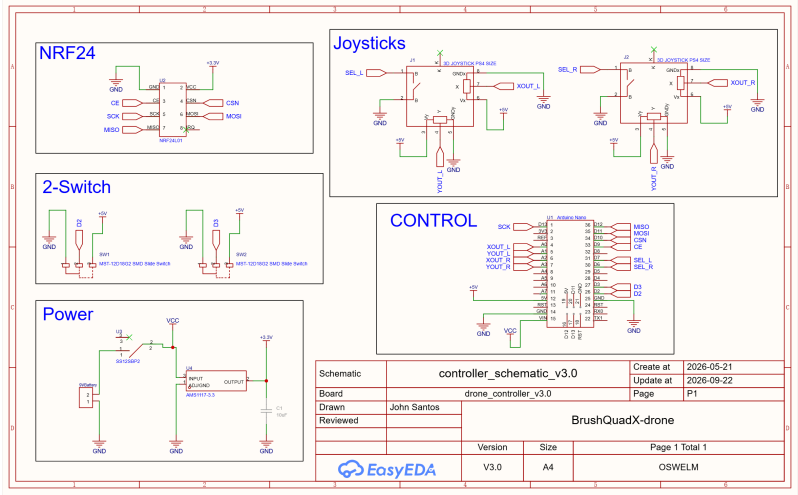
This page will show the step-by-step tutorial for building the BrushQuadX controller.

3.1 Materials

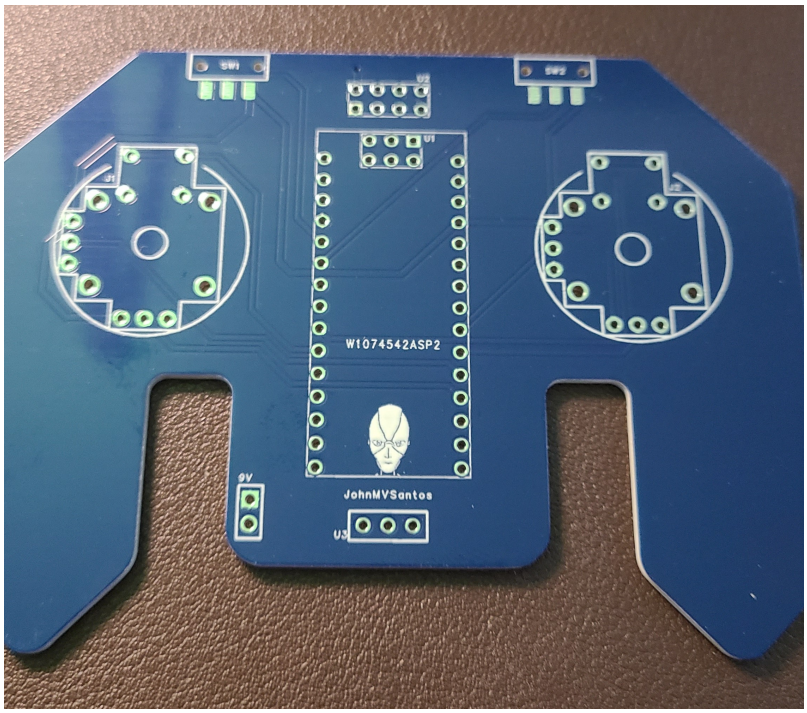
1. Arduino Nano Atmega328P (x1)
2. NRF24L01+PA+LNA Transceiver Module with 8-pin Breakout Adapters (x1)
3. Joystick Module for Arduino Dual Axis Sensor (x2)
4. MST-12D18G2 SMD Slide Switch (x2)
5. SS12SBP2 Power Switch (x1)
6. AMS1117-3.3 Voltage Regulator (x1)
7. Charging Connector (x1)
8. 9V Battery (x1)
9. FPV Reciever 5.8G 150CH OTG USBC (x1)
10. 10uF capacitor (x1)

3.2 Hardware

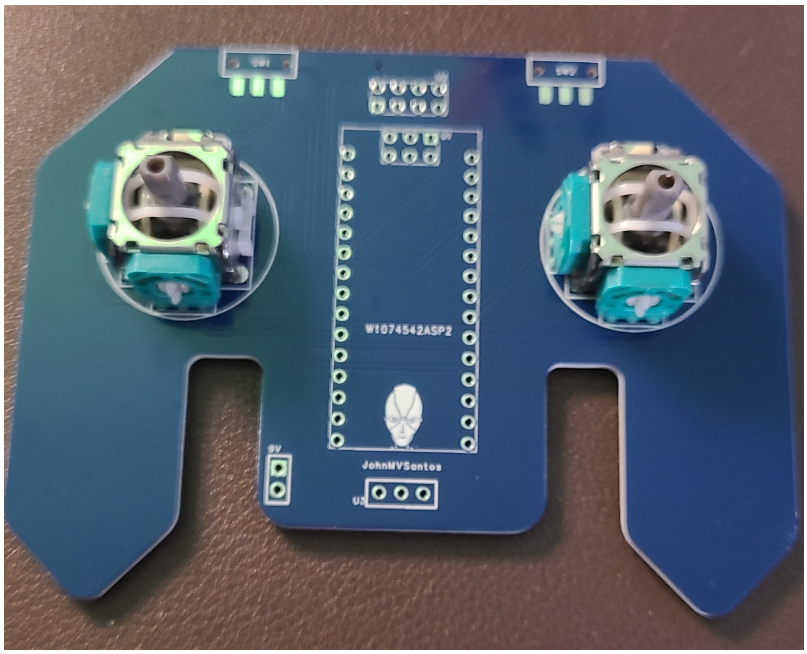
1. Take the Gerber files for the controller PCB and upload them to JLCPCB to manufacture the PCB. The Gerber file can be found in the *design_files/Gerber_Controller_PCB_v3.0_2026-10-02.zip*.
2. The controller PCB is based on the following schematic.



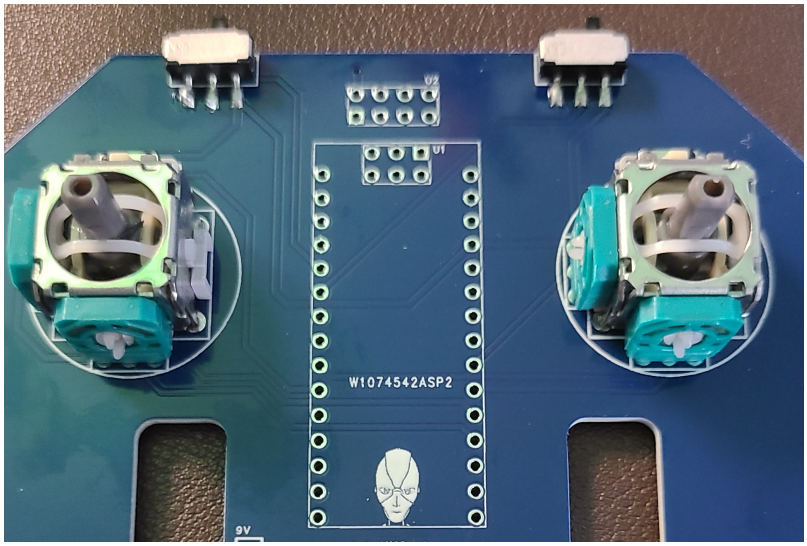
3. The PCB should appear as the following when it is received.



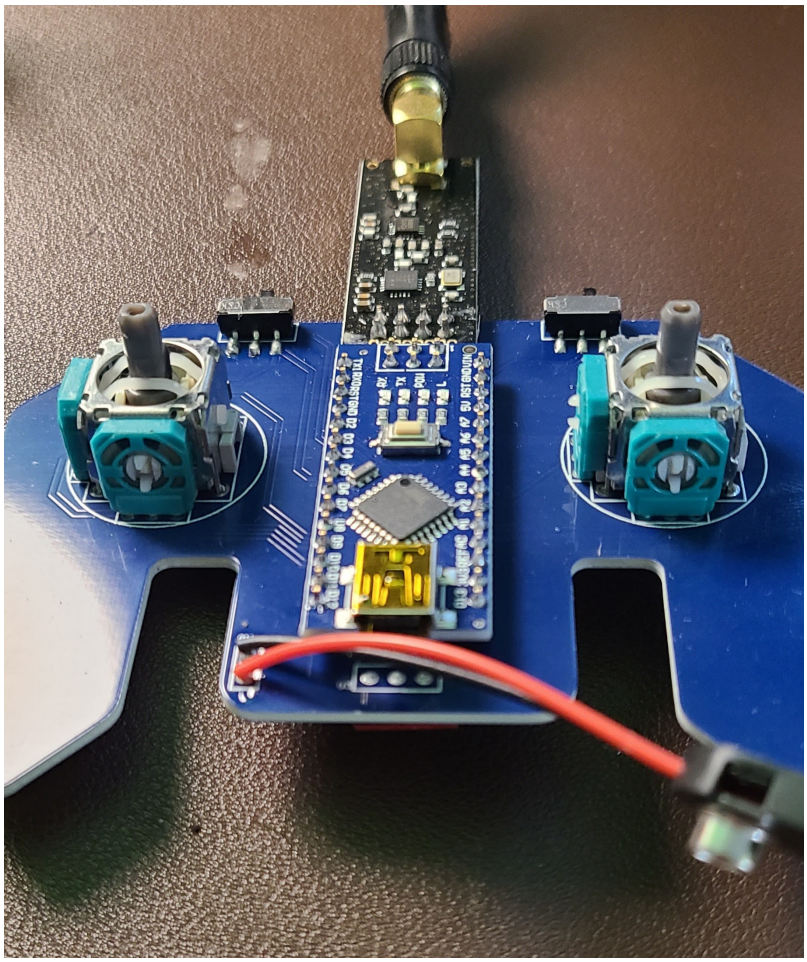
4. Solder the joystick modules to the PCB. It is recommended to remove the spring inside the joystick to avoid returning to the center position.



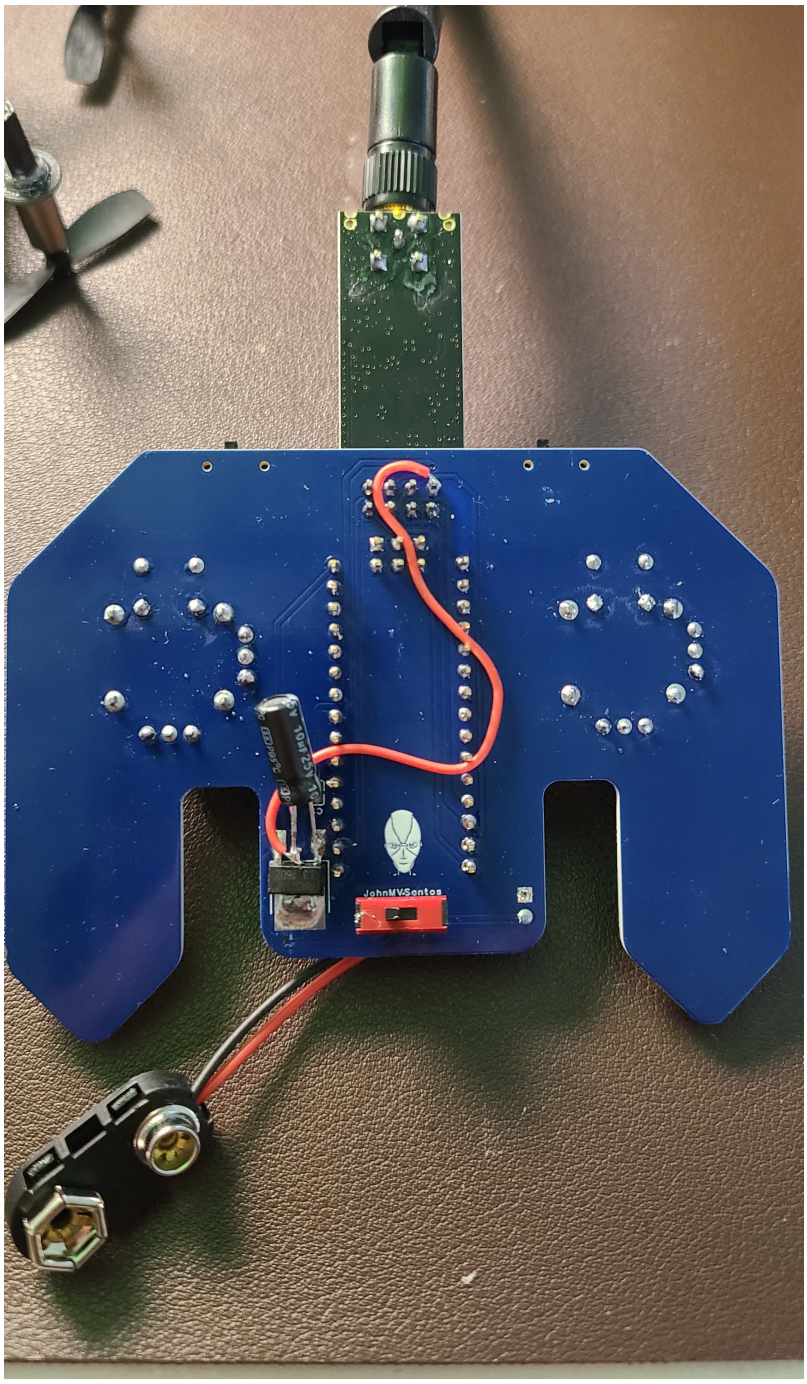
5. Solder the MST-12D18G2 SMD slide switches to the PCB. These switches will be used to control the flight modes (AUX1 and AUX2) of the drone.



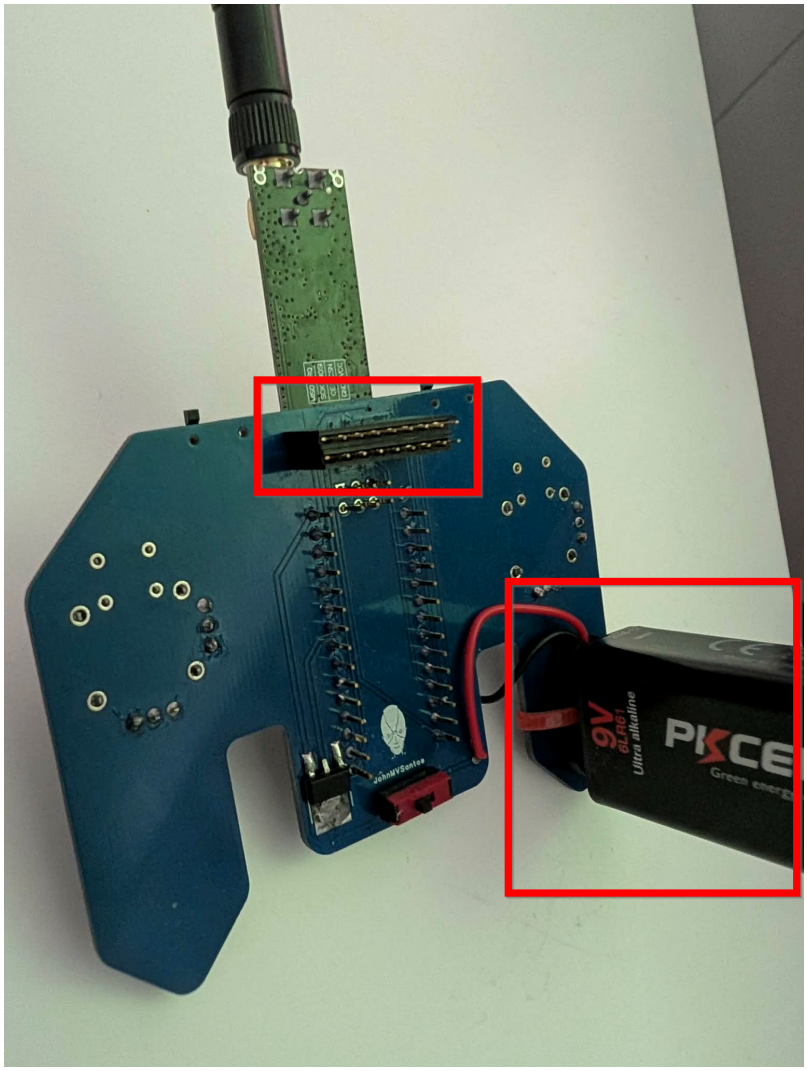
6. Solder the Arduino Nano and the NRF24L01+PA+LNA transceiver to the PCB as shown.



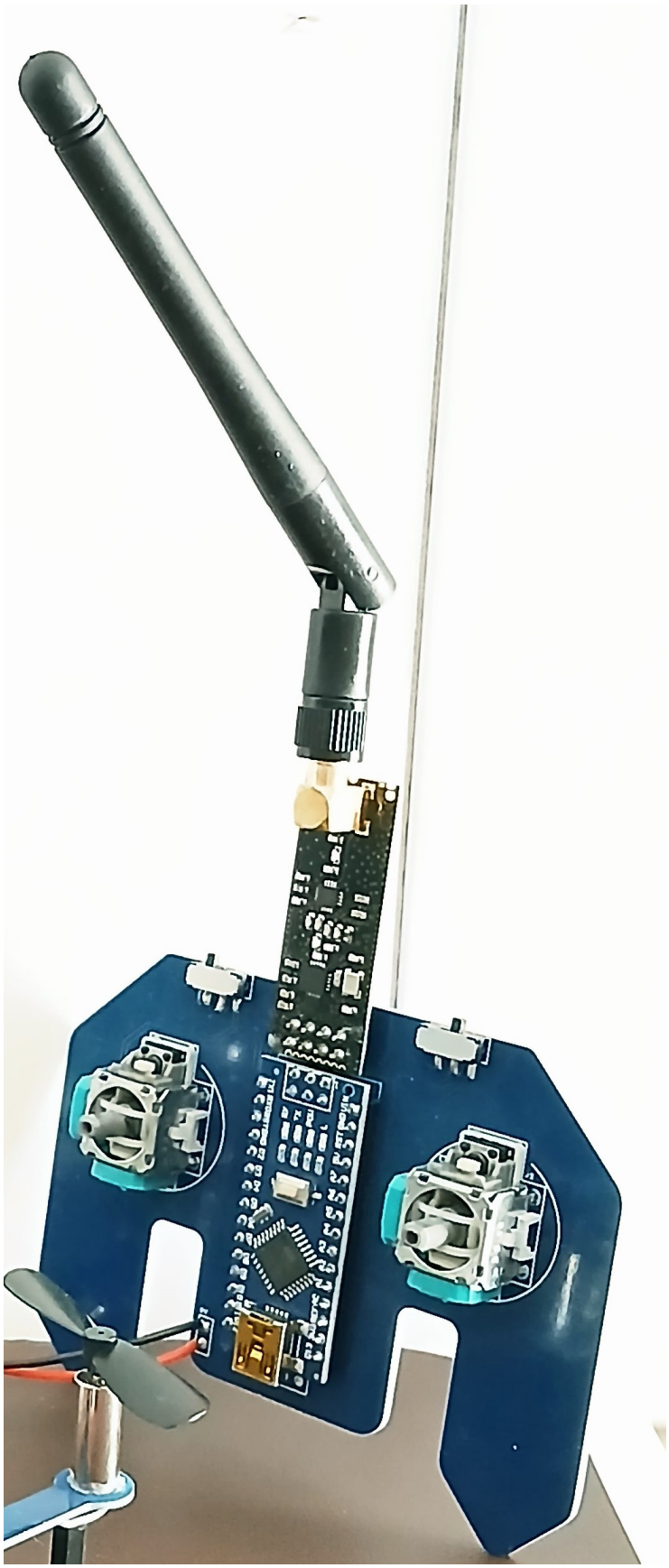
7. Solder the AMS1117-3.3 voltage regulator to the PCB with a 10uF capacitor. Then solder the SS12SBP2 power switch and the 9V battery connector to the PCB.



8. Add 8 PIN female straight headers (reduces noise) to the pinouts of the NRF24L01 to maintain the signal communication with the drone. Then wrap the 9V battery connector to the frame of the controller using a zip tie.



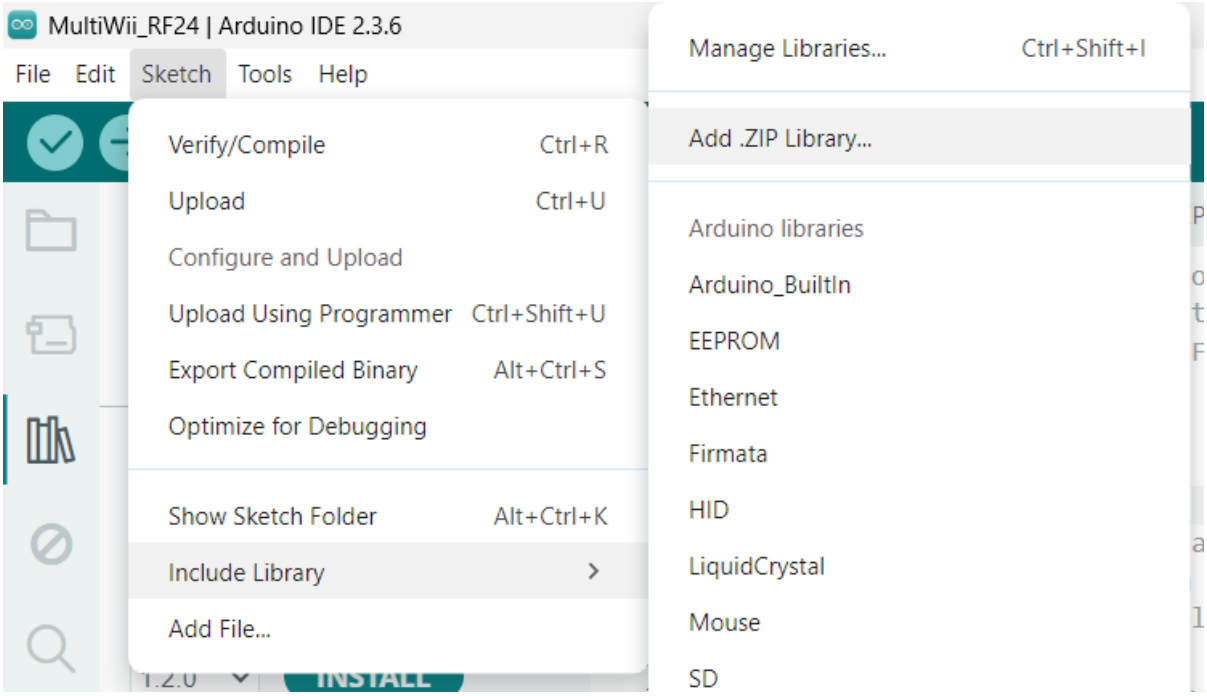
9. The assembled controller circuit should look like the following.



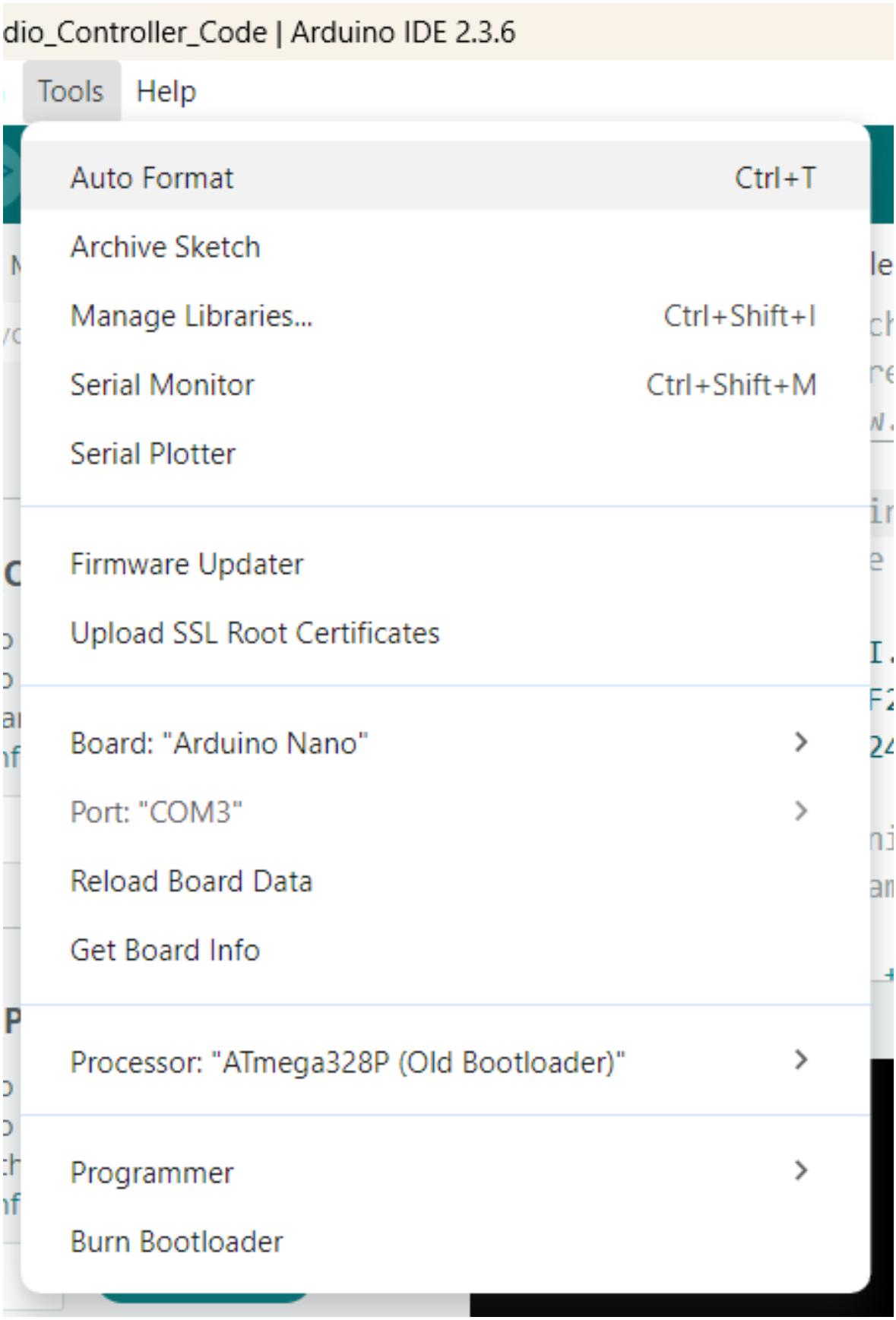
3.3 Software

1. Clone https://github.com/BrushQuadX/brushquadx-rf24-txcontroller repository.
2. Open Arduino IDE and open the project *BrushQuadX_RF24_TXController*.
3. Install the following libraries and include the ZIP libraries in the Arduino IDE.

- RF24
- TimerFreeTone_v1.5



4. Adjust the upload settings in Arduino under Tools to set the right board “Arduino Nano”, the COM Port, and the processor to “ATmega328P (Old Bootloader)”.


dio_Controller_Code | Arduino IDE 2.3.6
Tools
Help
Auto Format
Ctrl+T
Archive Sketch
Manage Libraries...
Ctrl+Shift+I
Serial Monitor
Ctrl+Shift+M
Serial Plotter
Firmware Updater
Upload SSL Root Certificates
Board: "Arduino Nano"
Port: "COM3"
Reload Board Data
Get Board Info
Processor: "ATmega328P (Old Bootloader)"
Programmer
Burn Bootloader

CHAPTER

FOUR

CONCEPTS

4.1 Metal Oxide Semiconductor Field-Effect Transistor (MOSFET)

The MOSFET is a type of transistor that controls the amount of current flowing through a circuit. It has 3 terminals: the gate, the drain, and the source. The gate terminal controls the flow of current between the drain and source terminals. When a voltage is applied to the gate, it creates an electric field that allows current to flow from the drain to the source in an N-channel MOSFET.

The MOSFET can also be referred to as a switch that turns a signal on or off.

4.1.1 N-type Doping

N-type doping starts with a silicon crystal which is stable by having 8 valence electrons. Each silicon atom consists of 4 valence electrons, but through covalent bonding, the electrons are shared achieving 8 valence electrons.

Then N-type doping is done by injecting other elements such as phosphorus with 5 valence electrons to produce 9 valence electrons in the overall structure. This results in one net electron leaving the entire structure negatively charged.

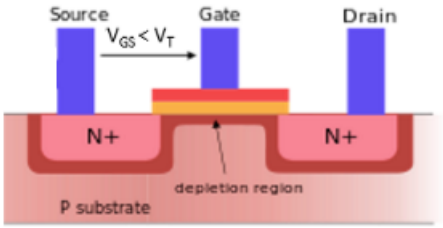
4.1.2 P- type Doping

P-type doping also starts with a silicon crystal and injected with an element such as Boron with 3 valence electrons. This produces 7 valence electrons in the overall structure. This results in a debt of one electron from a stable structure, leaving the entire structure positively charged.

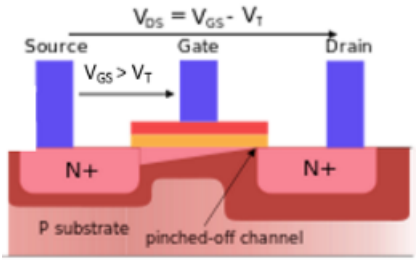
4.1.3 Design

An N-channel MOSFET consists of a base P-type material and an N-type drain and source materials. The N-type drain and source are connected to the P-type base material causing PN junctions to form where the two materials meet.

The PN junction forms when electrons from the N-type material move towards the P-type material. When electrons move towards the base and holes move towards the N-type material, an electric field forms preventing further the movement of charges. The PN-junction is also known as the depletion region. The region where mobile charges are absent.



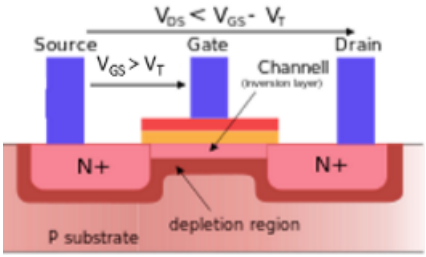
For an N-Channel MOSFET, the source is connected to ground. When voltage is applied from source to drain, there is no current flow at the source because it is grounded, but current flows to the drain. Adding more electrons to the drain, increasing the depletion region at the drain. The cutoff voltage is the voltage which current does not flow towards the drain.



Saturation at pinch off

The gate is separated from the base with a Silicon Dioxide layer. This layer has an extremely high resistance preventing current from flowing through the gate.

When voltage is applied at the gate, an electric field forms acting as a capacitor. This electric field creates a positive terminal at the gate, thus attracting electrons from the base to the top of the structure forming a channel from source to drain. As the voltage at the gate increase this forms a large enough channel for electrons to flow from source to drain. The voltage which allows current to flow is the threshold voltage.

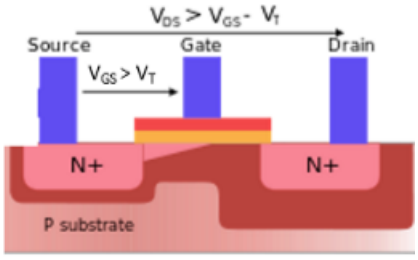


Triode

The gate voltage must be connected back to ground to introduce electrons at the gate thus dissipating the channel from source to drain.

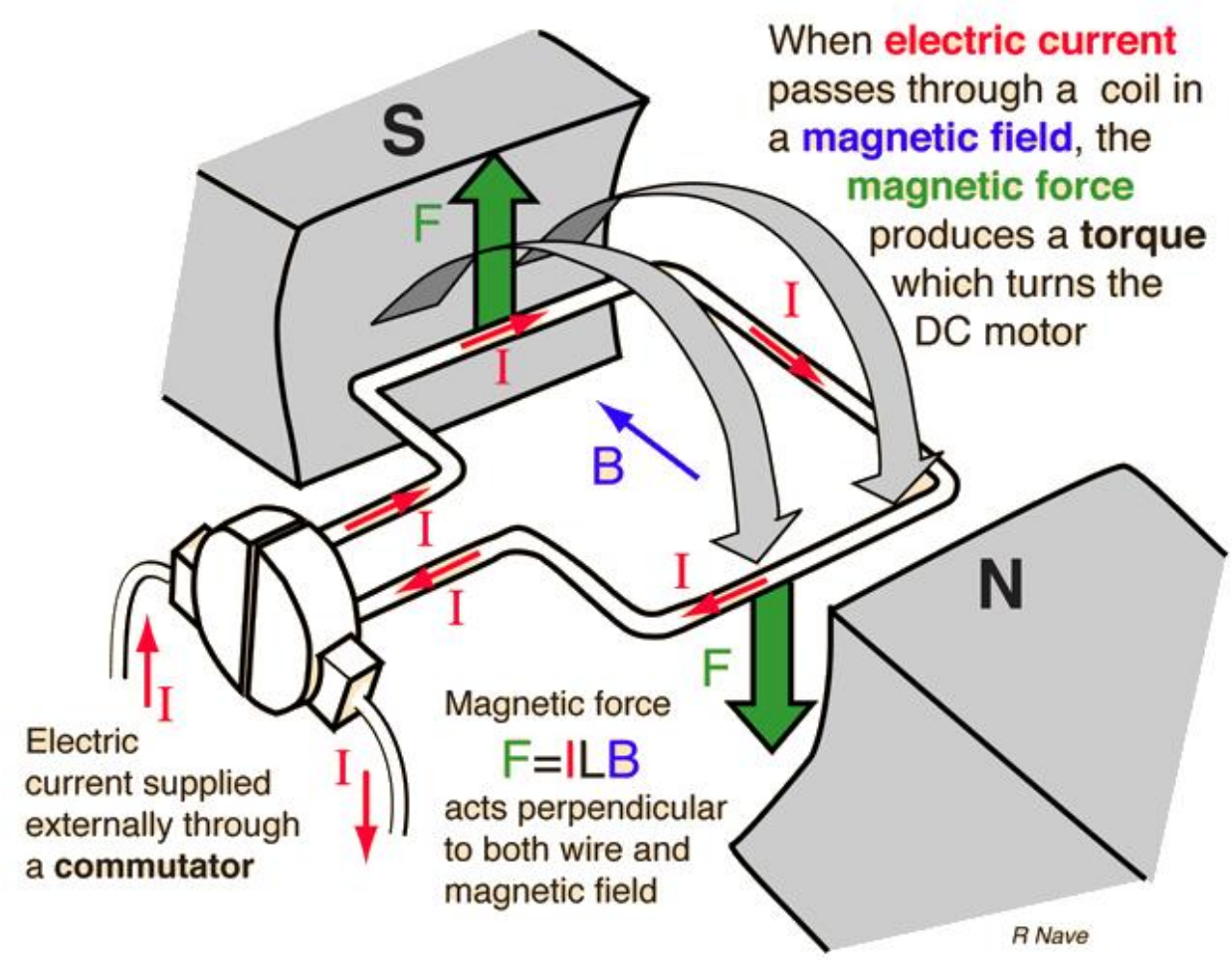
The point at which the channel is at the maximum capacity is the Ohmic region. Eventually current stops increasing at the saturation region at the specific voltage. Current can only be increased by increasing voltage.

The saturation region occurs when an electric field forms because of the voltage that forms from drain to source (VDS). This voltage is formed because of the moving electrons from source to drain. The electric field pushes the electrons from source to drain thus increasing the depletion region (reducing channel size) near the drain.

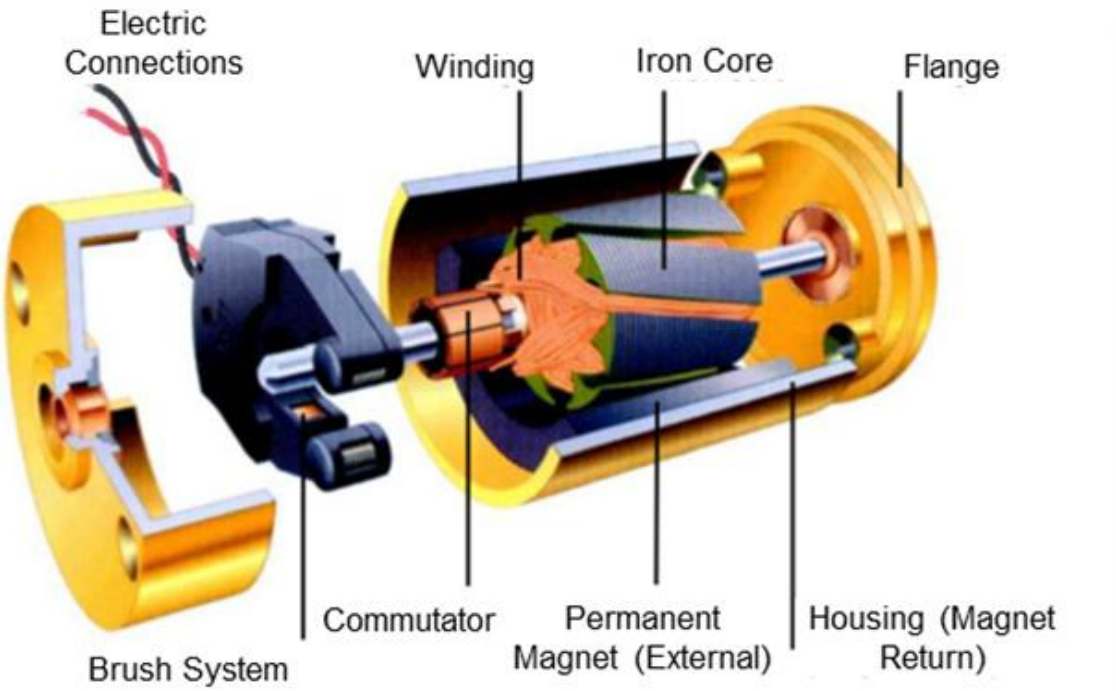


Saturation

4.2 Motors and Lenz' Law



A brushed DC motor generates torque through the interaction between the magnetic field produced by current flowing through the armature coils and the magnetic field of the permanent magnets.



1. Stator (permanent magnets) : provides stationary magnetic fields from the north and south poles.
2. Rotor (armature) : consists of an iron core and copper windings - the current flowing through the windings produces a magnetic field.
3. Commutator : segmented copper ring that reverses the current direction at every half turn to flip the direction of the magnetic force rotating the motor. The commutator connects with the armature and contacts the brushes.
4. Brushes : conductive contacts that transfer electrical current from the power supply to the rotating commutator.

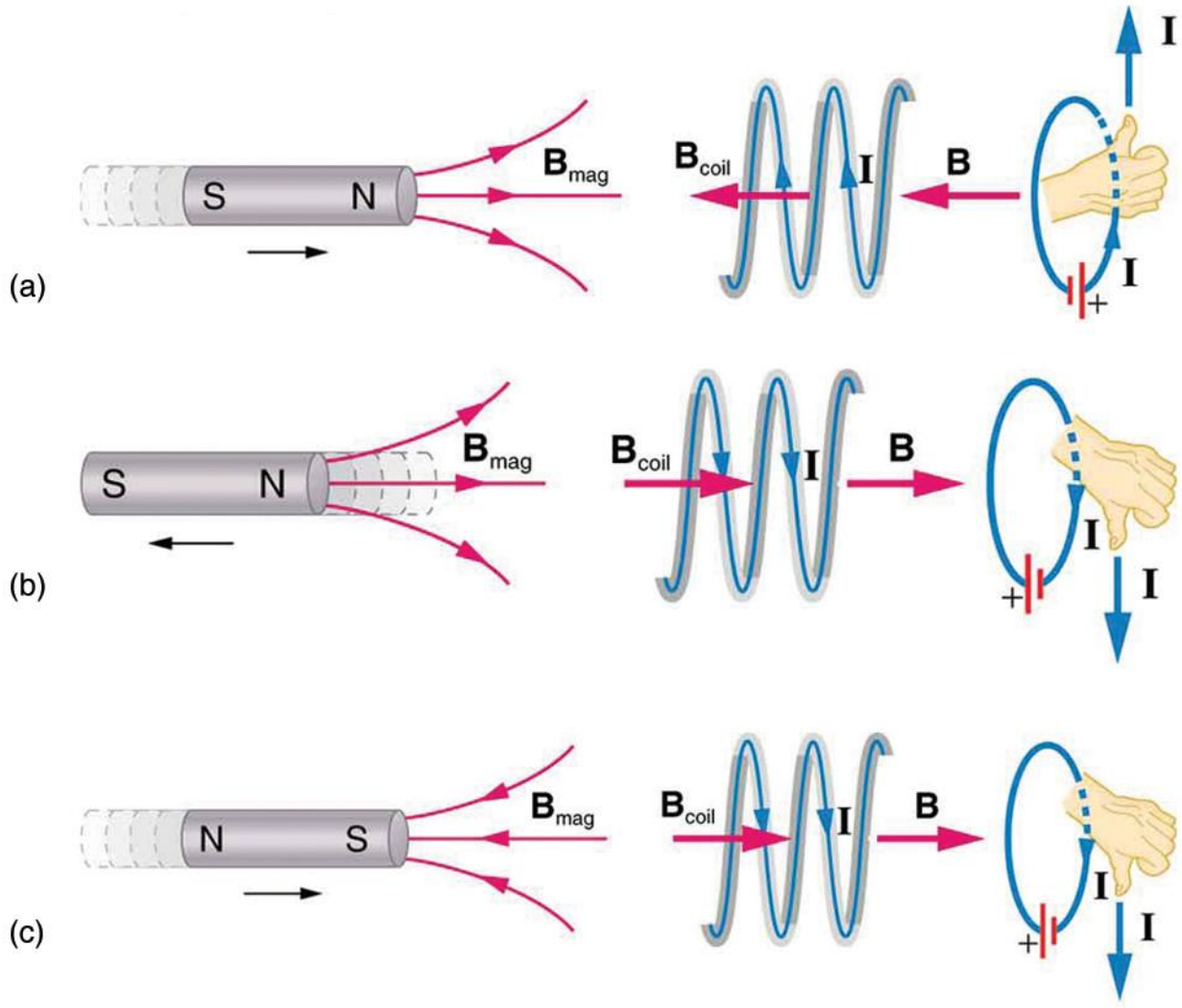
Motors are inductive loads as represented by this equation:

$$V = L\frac{di}{dt}$$

This means that counteractive forces are introduced to reduce the changes of the system and preserves conservation of energy.

- Electromagnetic Induction: the production of a voltage (EMF) when the magnetic flux through a conductor changes.
- Lenz' Law: The induced current flows in a direction that produces a magnetic field opposing the change in magnetic flux that caused it.

An induced current is caused by an induced electric field which is created by a changing magnetic field.



A changing magnetic field induces a circular electric field. When the MOSFET is turned off, this can induce an electric field which can generate large voltage spikes and "flyback" currents that can damage the MOSFET. This makes the design of the motor driver important to introduce protective components that keeps the MOSFET away from harm.

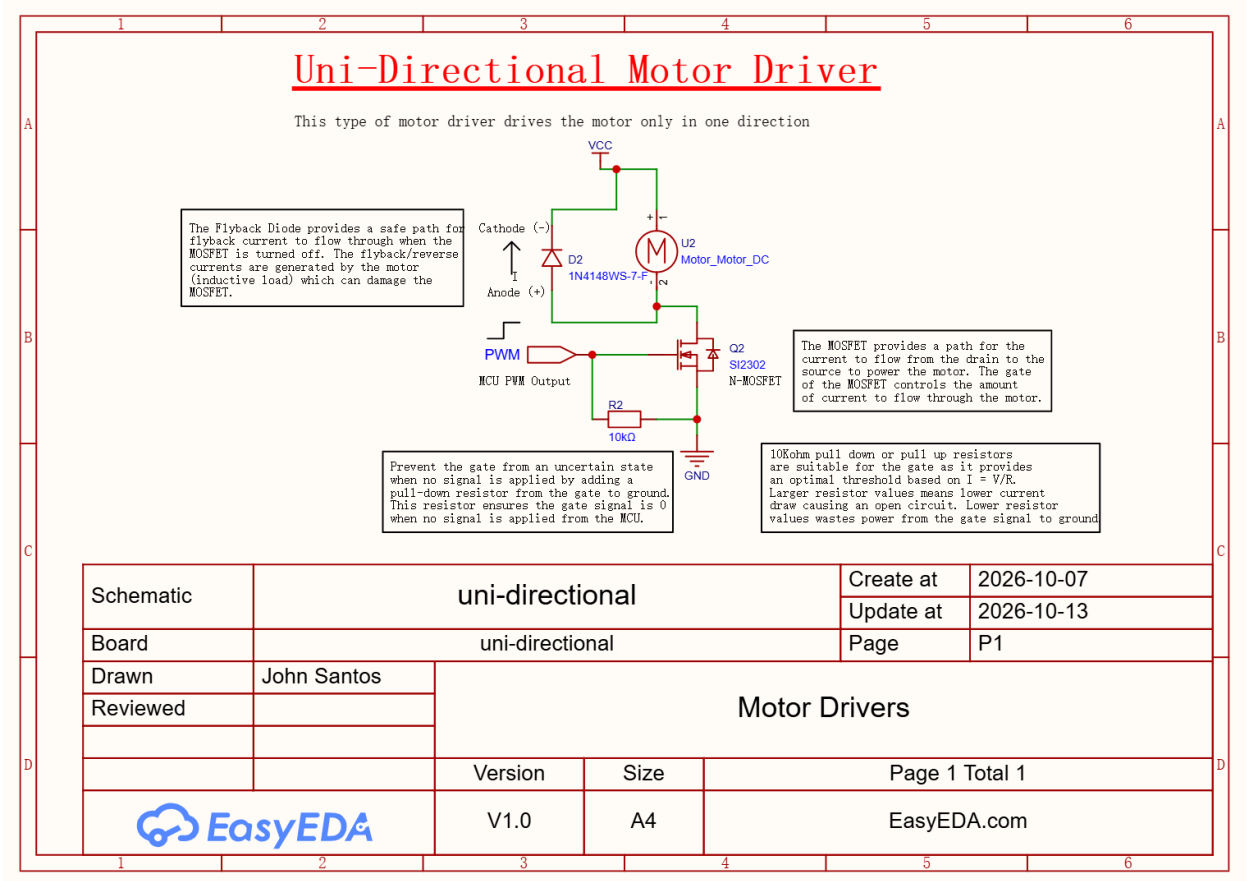
Motors drivers provides the following advantages:

- High current switching
- Flyback protection
- Overcurrent protection
- Thermal protection
- PWM control
- Direction control
- MCU pins does not supply enough current to drive the motors, motor drivers uses a separate power supply to drive the motors

Choose a set of motors whose stall currents won't exceed the maximum current rating of the motor driver. The stall current can be measured by applying maximum voltage to the motor and gently stopping the motor shaft using a set of pliers to see the current measured.

4.3 Uni-directional Motor Driver

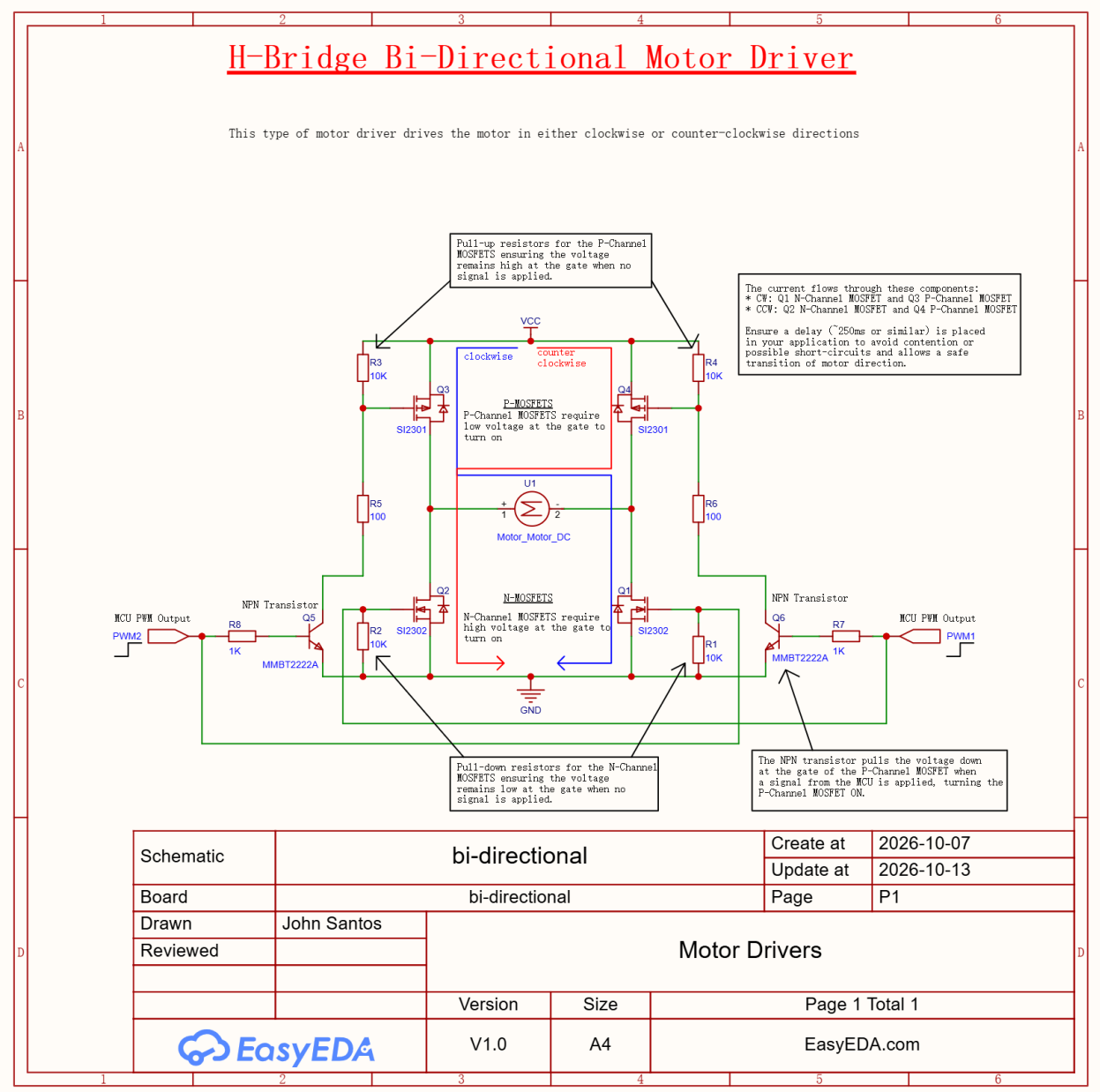
These types of motor drivers drive the motors in one direction only. They are typically used in applications where the motor only needs to rotate in a single direction, such as in conveyor belts or simple robotic arms or RC drones/planes.



This motor driver includes a flyback diode to drive the flyback/reverse currents generated when the MOSFET is turned off though the diode from the anode to the cathode to prevent any damages to the MOSFET. Furthermore, this motor driver includes a pull-down resistor at the gate to prevent any uncertain states when there is no signal at the gate. The pull-down resistor pulls the signal down to ground to ensure the gate at the MOSFET remains closed.

4.4 Bi-directional Motor Driver

These types of motor drivers can drive the motors in both directions (CW and CCW). They are commonly used in applications where the motor needs to rotate in both directions, such as in RC cars with forward or reverse mobility, bidirectional conveyor systems, or any application requiring forward and reverse motion.



This motor driver provides two paths for current to flow through the motor allowing the motor to spin in either direction. The path is as follows:

- Clockwise: Q1 N-channel MOSFET and Q3 P-channel MOSFET are turned on to allow the current to flow from the + to - terminals of the motor.
- Counter-clockwise: Q2 N-channel MOSFET and Q4 P-channel MOSFET are turned on to allow the current to flow from the - to + terminals of the motor.

We can never allow P-channel and N-channel MOSFETs on the same side of the bridge to turn on as this will cause a short circuit because of the absence of a load.

P-channel MOSFETs turn on when voltage is low and N-channel MOSFETs turn on when voltage is high. To allow the same signal from the MCU to control both types of MOSFETs, we can use a NOT gate (NPN transistor connected directly to ground) to drive the voltage at the gate of the P-channel MOSFET low when signal from the MCU is high, thus turning the P-channel MOSFET on. The signal from the MCU can be connected directly to the N-channel MOSFET as these types of MOSFETs will turn on when signal is high.

Pull-down resistors are placed at the gate of the N-channel MOSFETs to avoid uncertain states when no signal is applied at the gate. This ensures the N-channel MOSFET remains off. Similarly pull-up resistors are placed at the gate of the P-channel MOSFETs to ensure the MOSFET remains off when no signal is applied.

4.5 PWM Signals

Why use PWM signals to the gate pin of the MOSFET? PWM signals allows us to efficiently provide average motor power. Let's define the following DC logic levels.

- 5V -> MOSFET is on (motors at 100% fully powered)
- 0V -> MOSFET is off (motors are not powered)

What if we want to power the motor only at 50%? If we pass 2.5V to the gate, then the MOSFET is 50% ON. Though this seems logical to achieve 50% motor power, but it is highly inefficient. A MOSFET that is not fully open adds resistance to the current which can result to wasted power.

$$P = I^2R$$

Let's say the internal resistance is 1ohm at the MOSFET powered at 2.5V, this results in a power loss of 1W. However, if the internal resistance is 0.01ohm at the MOSFET powered at 5V, this results in a power loss of only 10mW. A fully powered MOSFET is very efficient, and in order to achieve 50% power to the motor, we provide a 50% duty cycle to the gate of the motor. This means that the MOSFET switches ON for 1/2 of the period and OFF for the other 1/2 of the period. Thus on average, the motor is only receiving 50% of the power from the power supply for any given time.

Voltage is supplied at the gate because the gate acts like a capacitor to control the MOSFET. High internal resistance prevents any current from flowing through the gate.

Furthermore, if the PWM is set to 20KHz, this doesn't mean that the motor instantly accelerate or decelerate 20,000x per second. Mechanical inertia smoothes the motor into continuous rotation.

CHAPTER

FIVE

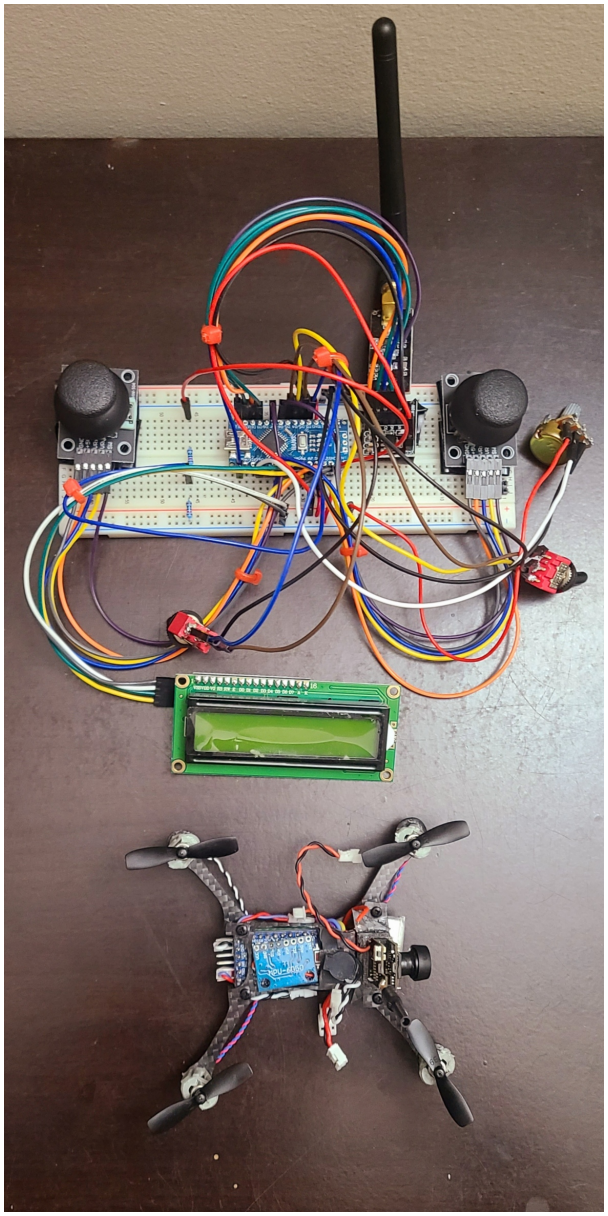
REFLECTION

The inspiration behind this project stems from Max Imaginations' Mini FPV Drone Tutorial. Watching his video sparked my interest in recreating the project with my own design and modifications. Furthermore, I wanted to apply the technical concepts I learned throughout my studies in electrical engineering to enhance my understanding of the subjects.

It was not easy to commit to the project in the beginning because I knew that I will be spending lots of time and resources towards re-design and re-iteration and acquiring electrical and mechanical concepts. Furthermore, I understood that there will be new concepts that I'll learn outside of my studies in University such as designing a custom PCB board. On top of this, my time is even more limited because I'm currently committed to working full-time at a software company. So most of my time working on this project was spent during the weekends and late nights after work.

However, I was determined to see this project through completion; to have a better understanding of concepts related in electrical engineering and to create something new despite the challenges and my lack of personal experience in building drones.

5.1 Protoype 1 - June 8, 2025



The start of the project was slow as I had to first list all of the electrical and mechanical components I needed to build the drone and the controller. I researched the cost, size, and weight of each component because I wanted to design a custom frame for the drone. Although most of the components I used remains the same as Max Imaginations' drone, I wanted to modify the frame to appear more modern and modular using carbon fiber as the main material. So I sketched a design to my liking and drew a blueprint cutout of the various parts, and using a dremel, I cut out the pieces on a 0.5mm carbon fiber sheet. The pieces were assembled together using M2 screws and nuts to hold the pieces in place.

Next I redrew the circuit schematic using EasyEDA and purchased the various electrical components from Amazon

and Digikey or Mouser. Once all the components arrived, I started soldering the connections between the components using 24AWG wires. I had some challenges initially when trying to solder these components because the soldering iron oxidized quickly which is probably due to poor techniques because of my lack of experience or the solder material I used was poor. So I ended up switching the solder tips quite frequently. In the future, I had speculated that failures of certain components stems from the poor solder job which could either be from a cold solder (no effective contact) or smaller surface mounts had a short circuit.

I reached the stage where the drone and the controller was fully assembled and programmed using Arduino. However, I began seeing several issues when arming the drone using MultiWii; the drone's IMU movements and the controller's joystick movements were not captured and simulated in MultiWii which prevented the drone from being armed. Furthermore, MultiWii was only able to capture the IMU's movements when grounding pin D12 on the Arduino Pro Mini MCU which was quite arbitrary for me at that moment. Given the numerous unexpected behaviours and issues, I decided to take a step back and reflect on my design and assess the changes and corrections I needed to make to resolve the issues I was facing.

The main issues are listed below:

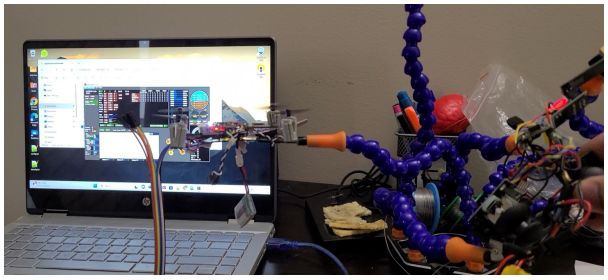
1. **Too heavy**
    - Motor fasteners that were improvised using drywall anchors were too heavy (motors will be glued directly in the next prototype)
    - The wires were too thick using 24AWG
    - The perforated board was too large with lots of unused space
    - The soldered JST connectors (NRF24L01, perforated board, buzzer) are too heavy (these will be removed and wires will be soldered directly in the next prototype)
    - 1N5819 diodes are too heavy (using 1N4148 surface mount diodes in the next prototype)
    - Using velcro to mount the camera is too heavy (use super glue instead)
2. **Small Propellers/Less Thrust**
    - Using 2 blade (faster) 37mm propellers (use 4 blade propellers for more thrust)
3. **Conductive Carbon Fiber Frame**
    - Possible short circuits when mounted on the carbon fiber frame (use kapton tape for better insulation with the electrical components)
4. **Poor Solder Connections**
    - Damaged solder tips (oxidized) resulted in poor solder connections with possible decrease in conductivity and connectivity between components and possible short circuits
    - Replace solder tips and properly resolder the connections in the next prototype
5. **Need Soldered Controller with Enclosure**
    - The controller uses a breadboard with weak connections and unmanaged wiring. Requires a proper enclosure with soldered connections for better connections

Additional materials and replacement needed for the next prototype:

1. Arduino Pro Mini Atmega 328P 5V/16MHz (un-soldered)
2. MPU6050 from DFRobot (un-soldered)
3. **Motor Encoder Circuit**
    - Surface mount 1N4148 diodes
    - Solder tips + high quality thin solder

4. Larger 4 blade propellers
5. Kapton tape for insulation
6. Copper Sheet for proper grounding

5.2 Protoype 1.1 - June 22, 2025



This prototype has a more compact design with shorter connections and lighter wires. Furthermore, the controller now has a proper enclosure created from a toy RC enclosure that has been modified to fit the components of this drone. However, despite the modifications, the drone was still not successful in lifting off.

This prototype started by assembling and soldering the connections between the eletrical components using 28 AWG silicon copper wires. The controller was built by modifying an old RC toy enclosure using a dremel tool so that the new components fit properly inside the enclosure.

Once the drone and the controller were assembled, both were then programmed using Arduino. In this prototype, MultiWii was able to capture the drone's IMU measurements. However, MultiWii still could not capture the controller's joystick movements. It turns out the radio transmitter I used for my controller was faulty and I learned that it is always best to test individual components first prior to integration into the system. A simple test can be setup to program two radio modules as a transmitter and a reciever to test whether or not both modules can properly communicate with each other. Once I received a new set of functional radio modules, the drone and MultiWii can finally receive the radio signals from the transmitter.

Lastly, I observed the drone's behaviour to be erratic; I was not fully confident that the drone's enclosure is properly insulated which may have affected the drone's components. So I tested the drone without the enclosure and I found that the motors seem to respond properly when I throttle up and down using the controller. Yet the issue of poor insulation on the carbon fiber enclosure remained and I had a speculation that the drone short circuited from the enclosure after reattaching causing permanent damage to the electrical components. I learned that I should have tested the insulation of the frame and should not have relied on the assumption that the kapton tape I applied had fully insulated the enclosure.

Yet again, this prototype has failed so I have taken another step back to assess the failures and make further revisions to resolve the issues.

The main issues are listed below:

1. **Kapton Tape did not provide proper insulation**
    - After adding one layer of kapton tape, it seemed that the electrical components, mainly the IMU shorted and was damaged. This resulted in the accelerometer unable to properly register the movements in MultiWii. The next prototype will include a proper enclosure for the electrical components of the drone. Although the main frame will still be carbon fiber, a separate box enclsoure will be used to cover the electrical components properly using light materials
    - Investigated the use of hot glue for proper insulation; however, initial research suggests this may cause damages to the electrical components
2. **Controller is Too Complex**
    - "Simplicity is the ultimate sophistication." - Leonardo da Vinci

- Unnessary components in the controller atleast for the MVP such as the 16x2 LCD to track voltage and the potentiometer should be removed. The controller should be simplified to only include the basic components needed for communicating with the drone and this includes the Arduino Nano controller, the radio module + PA + LNA components, the two joysticks, the two SPDT switches, and the SPST switch with the 3.7V batteries

3. **Controller Battery has too much current**

    - Prolonged use of the controller lead to overheating of the components and controller failure. Theory is that the battery packs too much current which the components could not handle resulting in break-down. The two batteries are connected in series are 3.7V 1000mAH. Looking into the use of 3.7V and 600mAH batteries instead

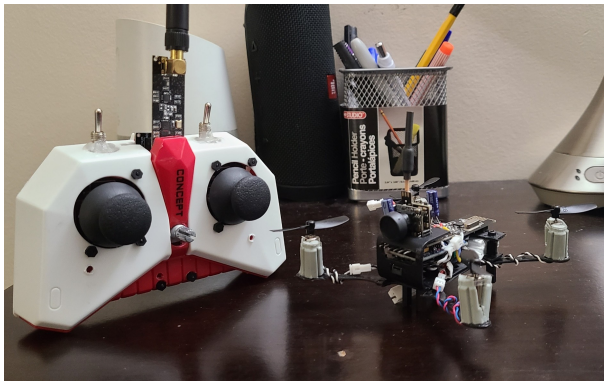
4. **Remove the Grounded Copper Sheet**

    - This may not be needed as I have not encountered any issues with the drone resetting. This solution was suggested online, but I should not implement solutions to problems that does not exist in my design

Additional materials and replacement needed for the next prototype:

1. Arduino Pro Mini Atmega 328P 5V/16MHz (un-soldered)
2. Arduino Nano
3. Radio Modules NRF24L01 + PA + LNA
4. 3.7V 600mAH batteries (2x)

5.3 Prototype 1.2 - August 18, 2025



The behaviour of this prototype was still erratic; the drone kept resetting and recalibrating when I throttle down. The motor responses were rare and seldom spins. When the motor does respond, it respond in bursts and then the drone resets. Looking at the MultiWii outputs, the AUX1 and AUX2 switches and the joystick movements for roll, pitch, yaw, throttle levels were correctly simulated in the software. So I think the issue stems from the drone hardware rather than the controller.

The primary issue in this prototype is that the motors do not respond despite the joystick movements being translated correctly in MulitWii. The drone keeps resetting and attempts to calibrate everytime the controller attempts to throttle up and down.

I suspect the possible issues with the drone are listed below:

1. **The battery discharge rate is too low (25C) and that a proper drone battery with a higher discharge rate (30C or higher) is needed**
    - *Recommended to use "Turnigy Nano-Tech" batteries or similar for their high performance*
2. The power for the radio is not consistent and requires a 10uF filtering capacitor at the NRF24 power inputs
3. **The power for the Arduino Pro Mini is not consistent and requires a 100uF filtering capacitor**
    - Confirm Arduino Pro Mini 3.3V 8MHz, or 5V 16MHz is required (*Arduino Pro Mini 5V 16MHz is compatible with MultiWii*)
4. **The power lines has a large AWG (small thickness) where the current cannot be supplied properly**
    - Recommended to use solder with lead and keep solder enclosed after use to avoid contamination/oxidation
    - For the motor driver, ensure the components are rated for this circuit
    - The following components are being used but requires confirmation; 0603 10K SMD resistor 103, SI2300DS-T1-GE3CT-ND N-Channel Mosfet 30V 3.6A, 1N4148 diode surface mount
5. **Motor PWM signals could be too weak to drive the motors**
    - Requires oscilloscope to confirm gate frequency
    - This factor can be set in the drone firmware, *float adjustmentFactor* on line 1069 of output.cpp
6. Potential EMF noise or leaks is affecting the IMU readings?

7. The Arduino Pro Mini purchased from "Hutomwua" is faulty. The previous prototypes purchased from "Robojax" was working

Looking for a battery replacement greater than 25C was quite a challenge as most of the batteries I found were 25C or lower. It was recommended to use "Turnigy Nano-Tech" batteries for their high performance, but those were always unavailable and kept being out of stock due to their high demand. Although I did manage to find a 30C LiPO battery from AMZZN, but it still did not improve the drone's behaviour.

Additional materials and replacement needed for the next prototype:

1. Arduino Pro Mini Atmega 328P 5V/16MHz (un-soldered) from Robojax specifically

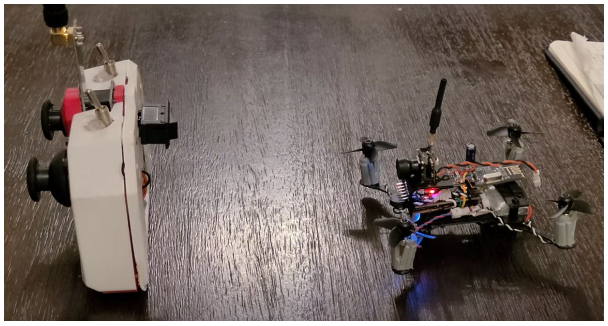
I then tried adding filtering capacitors at the power inputs of the NRF24L01 and the Arduino Pro Mini, but this also did not improve the drone's behaviour. I also ensured the power and radio connections were solid and properly connected. Next I created a new motor driver with new components as I had speculated the previous build had short circuited, but this new replacement still did not improve the drone's behaviour.

I then suspected the issues had to do with the drone's software and the PWM signals driving the motors had a low frequency. Unfortunately, I did not have an oscilloscope to verify the PWM signals at the gates, but I found the factor in the drone's software that tunes the PWM outputs. By following the tutorial in Max Imaginations' video, the variable that adjusts the PWM signals is controlled by *float adjustmentFactor = 0.255* in line 1069 of output.cpp. The modifications made to this variable did not improve the drone's erratic behaviour, but it did tune the buzzer's noises lower. As a future work, it seems best to purchase an oscilloscope confirm the PWM frequency suspicions.

My final attempts in resolving the drone's erratic behaviour was retracing my steps for properly arming the drone using MultiWii. First the AUX1 and AUX2 switches should be set high when arming the drone and the beeper. Once set, the drone accelerometer should be recalibrated an then press read and write to save the changes. To arm and recalibrate the drone, the AUX1 switch should be set low and the throttle and pitch joysticks should be moved to zero until the calibration signals from the beeper are in effect. Once in effect, wait atleast 3 seconds to apply the recalibration. By following these steps, the drone still did not manage to respond from my controller's commands.

My suspicion towards the source of the issue is the drone's hardware. The Arduino Pro Mini I used might have been faulty since the previous prototype (v1.1) did respond to my controller's commands. I will find another microcontroller ideally from the same manufacturer as v1.1 and then I will confirm the specifications of the motor driver components are properly rated for this design and meets the power requirements.

5.4 Prototype 1.3 - August 31, 2025



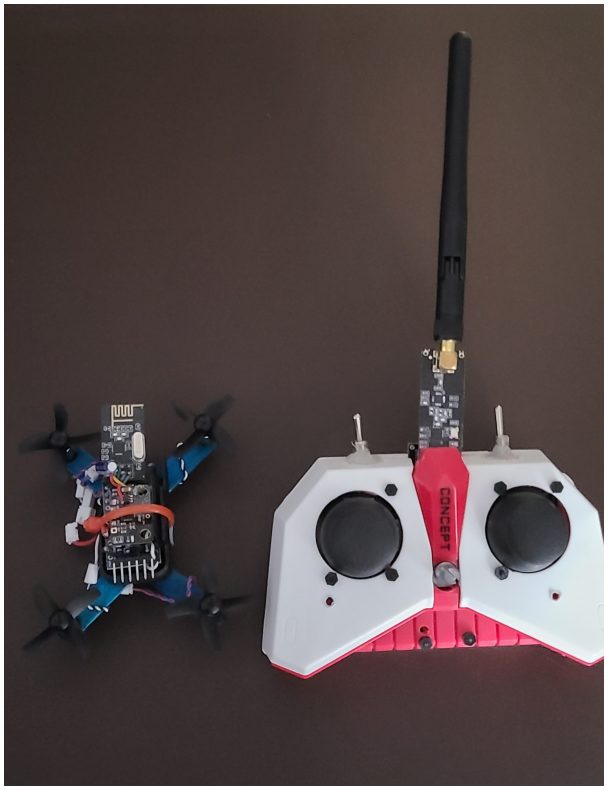
The suspicion that the Arduino Pro Mini microcontroller being faulty was true. The Arduino Pro Mini purchased from "Hutomwua" had a board frequency of 10MHz. The requirements for correctly interfacing with MultiWii requires a 5V 16MHz microcontroller. I re-purchased a 5V 16MHz Arduino Pro Mini from "Robojax" which resolved the erratic drone behaviour which allowed the drone to respond to the controller's joystick movements.

However, the design is still not perfect as more issues surfaced. Although the motors responds to the joystick movements, the drone's attempt to lift off was quite unstable.

The possible issues are listed below

1. **The drone is not calibrated properly**
    - The drone needs to sit on a flat surface for a proper calibration
    - The accelerometer and the gyroscope needs proper calibration
    - Adjust settings in MultiWii configuration with max smoothness
2. **Motor/propeller direction is wrong**
    - This can be verified by feeling if the air is being pushed upwards
    - Record slow motion video to see the direction of the motors
3. The motor RPMs are not the same and unsynchronized
4. **The forward direction of the MPU6050 is in the opposite direction**
    - Rewire orientation of the motors to have face the MPU6050 in its forward direction
5. **The drone is still too heavy and certain weights of the components are not balanced causing the center of gravity to be offset**
    - Remove heavy motor mounts and just rely on superglue to attach the motors
6. **The arms of the drone are limble which causes wobbling during flight**
    - Reinforce the arms with a stronger material that doesn't bend such as popsicle sticks

5.5 Prototype 1.4 - November 30, 2025



In this prototype, the drone accelerometer was re-calibrated and an EMF copper blocking shield was installed to prevent EMF interference generated from the motor driver to the Arduino Pro Mini microcontroller. Furthermore, the motor direction was verified:

- Top left -> clockwise
- Top right -> counter clockwise
- Bottom left -> counter clockwise
- Bottom right -> clockwise

The bottom right propeller from the previous prototype was wrong which pushed the air upwards. For stability, the arms were replaced with popsicle sticks to prevent any oscillations during flight and the motors were super glued directly to the arms.

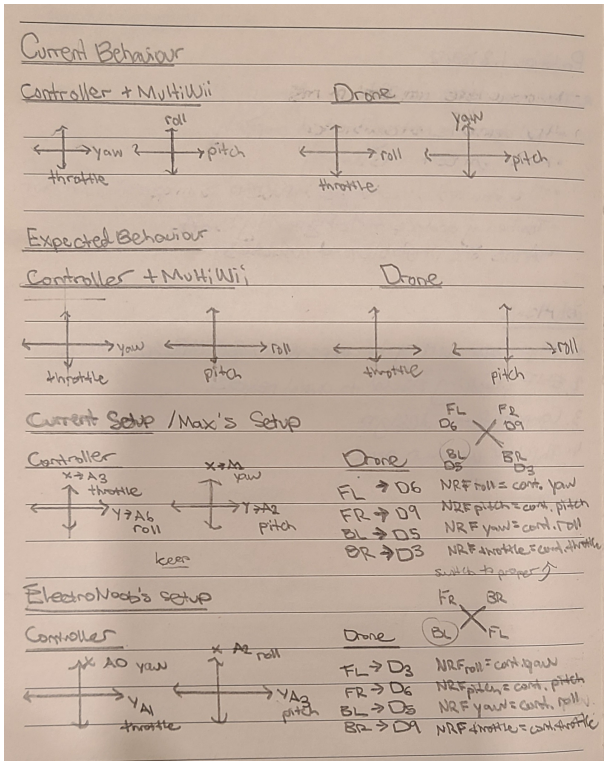
Despite the changes, the drone still could not achieve a proper liftoff. PID tuning has also been explored and adjusted, but I could still not find the right values for a stable flight.

The possible remaining issues are listed below

1. IMU is not secured properly and wobbles during flight
2. The frame is still too heavy
3. Smaller propellers + less surface area results in unstable flights

4. Battery is mounted at the bottom making it hard to connect - perhaps a switch is more convenient
5. Wrong drone output behaviour or motor mapping from the joystick movements - requires a software correction
6. Needs further PID tuning for the roll, pitch, and yaw values

The current configurations with the expected behaviour and the actual behaviour are noted below.

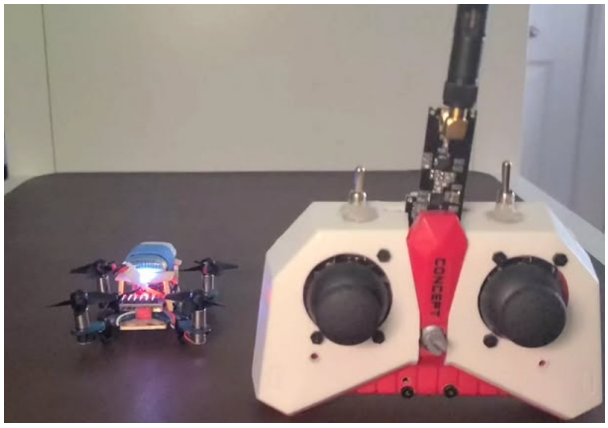


Design changes for the next prototype

New frame requires light and thinner materials, but maintain thicker inflexible arms with more propeller surface area. The battery should be mounted on the top for easier connection and allow center of gravity re-adjustments. Superglue connections instead of screws to reduce weight.

1. Software correction to the motor mapping behaviour
2. Configure joystick values for correct controller calibration
3. Lighter frame, longer arms, secure mounts
4. Proper PID tuning for the drone's roll, pitch, and yaw

5.6 Prototype 1.5 - January 31, 2026



This prototype is based on re-designing the frame to use popsicle sticks and superglue to reduce the overall weight of the drone. Despite the lighter frame, the drone is still unable to properly lift off. I suspect the issue could stem from the following reasons:

The possible issues are listed below

1. Components are placed that causes weight imbalance and an offset center of gravity - seems that the left side of the drone pulls up more
2. The motors spin at different RPMs causing unsynchronized motors - the PWM signals also needs to be verified
3. The motors still spins in the wrong direction
4. The drone also resets sometimes - a resistor at the reset pin of the microcontroller to avoid further resetting or it is missing a proper EMF blocking shield using a copper sheet with clear tape attached to a grounded wire
5. The drone might need PID tuning and joystick value adjustments to follow exactly Max Imaginations' settings
6. Motor driver components are too close to the controller (large EMF effects)
7. All power is drawn from the same source connected to the VCC pin of the Arduino Pro Mini preventing the microcontroller to receive sufficient power (perhaps power should be connected directly to the raw pin instead of VCC)
8. Wire thickness used is 28AWG (from 1.25mm connector sockets) - should be 30AWG+
9. The incorrect translation from the controller joystick movements to the drone motor behaviour is causing instability
10. IMU is not sitting flat and it is still unstable - potentially the IMU is not positioned correctly
11. Drone motors are not positioned evenly
12. The frame is still too heavy
13. Small surface area drone propellers is causing instability
14. Potential software issue settings are incorrect - try disabling *FORCE_GYRO*

Here is a list of unknown variables I encountered

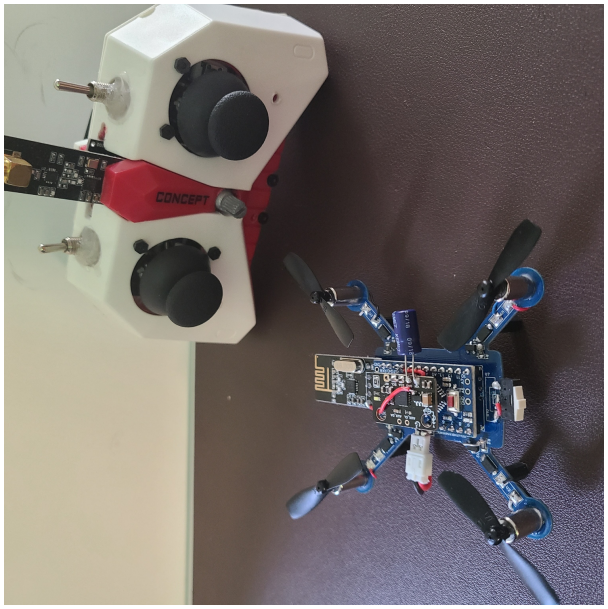
1. On MultiWii, the IMU yaw value continues to rotate even though the drone is placed flat on the table

Design changes for the next prototype

The next prototype will explore designing a custom made PCB to reduce complex external wiring; to have compact the design and reduce overall weight of the frame. Furthermore, I will explore different motor driver components to withstand potential current or power rating issues.

1. Placement of motor driver components on the arms to disperse combined EMF effects and reduce interference with the microcontroller
2. Power should be directly towards raw pin, VCC, or both?
3. Convert the drone to a PCB to have solid connections between components and reduce the weight from the wires
4. PWM drone pins should be consistent with Electronoobs: FR (D6), BR (D9), BL (D5), FL (D3)
5. Max Imaginations' right joystick seems rotated in his diagram - should also compensate my joystick to match his design
6. Calibrate controller settings to match the intended joystick min, middle, max values

5.7 Prototype 2.0 - March 28, 2026



In this prototype, a custom PCB for the drone and the controller was designed using EasyEDA and manufactured using JLCPCB. The motivation behind this decision was to produce clean and compact connections between components and ultimately reducing the overall weight of the drone. The PCB is designed to be its own frame and the components are soldered directly onto the board. I have chosen different motor driver components rated for this circuit.

- **10KOhm resistor 0805 (0.125W) CRG0805F10K**
  - Ensures VGS = 0
- **Diode: SS54FSH (5A, 40V)**
  - Maximum rated current must be greater than 2 * stall current (assuming 2-6A)
- **N- Channel Mosfet: AO3400A or AO3416**
  - VDS ~>20-30V is better (rated for high voltage spikes)
  - RDS ~<10mOhm is better (low power loss)
  - Gate threshold ~2.5-3.3V (turns on based on battery capacity)
  - Continuous & Pulsed Current Rating should exceed peak and continuous current draw in drone (5.5A = 25C * 0.22Ah)

The previous components had the following limitations:

- **10KOhm resistor 0603 (0.1W, 2.A max current)**
  - Low power and current rating
- **1N5819/1N4148 - Used: 1N4148WS-13-F (Consider using 1N5819 for low voltage drops)**
  - 1N5819: Low voltage drop (~0.3-0.4V)

  - 1N4148: Fast switching, but higher voltage drop (~0.7V)
  - Current rating must be greater than 2 * stall current (2-6A assuming)
- S12300 n-mosfet - Used: SI2300DS-T1-GE3CT-ND

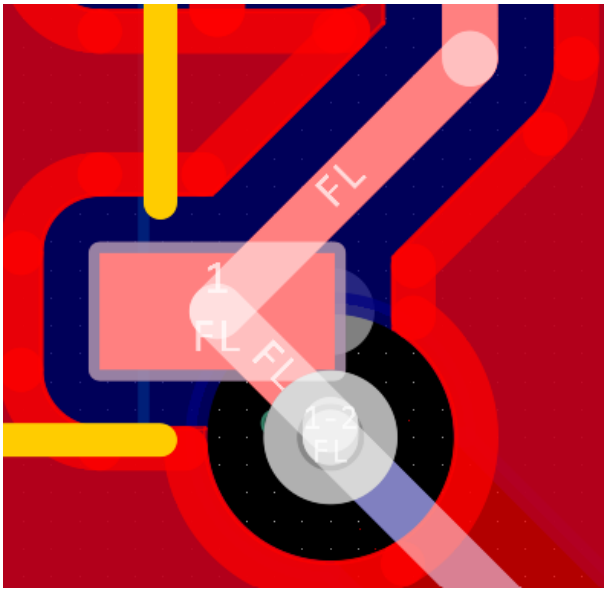
However, despite the design changes, the drone still could not lift off and behaves even poorly. When testing the assembled PCB, it seems most of the power is being drawn towards the back right motor since it is the only motor that spins. When I disconnect this motor, the other three motors spin, and the resetting issue from the previous prototype still persists. It seems that separating the motor driver across the arms of the drone does not mitigate the EMF effects towards the microcontroller since the drone continues to reset. I even tried adding 100uF capacitors between the power and the ground pins of the MPU6050, but this did not resolve the resetting issue.

The design issues discovered from this prototype are listed below

1. Bottom right motor keeps spinning uncontrollably - should separate power connections between the motors and the microcontroller
2. Battery with larger C rating did not fix the reset issue
3. 100uF capacitor at the MPU6050 power pins did not fix the reset issue
4. Thicker power lines and reinforcing connections with external wires did not resolve the issue
5. Speculating that MultiWii being an old software had problems arming the drone properly and my environment installations and Java versions could not support a stable MultiWii version

The possible reasons for these issues are listed below

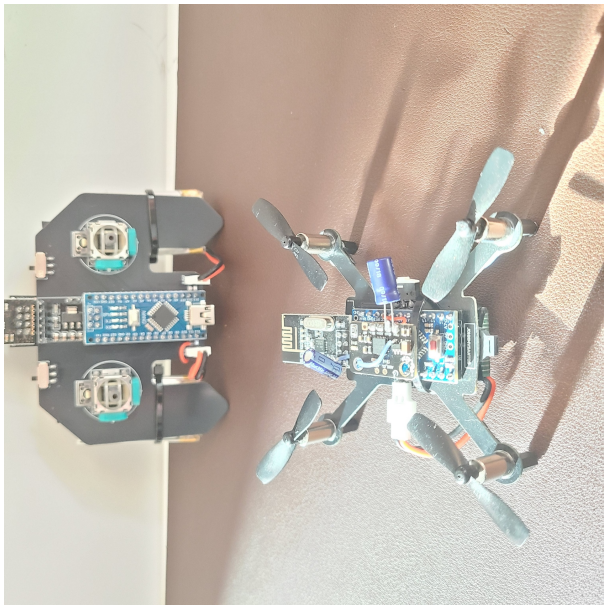
1. Power management - the battery is not strong enough to drive both the Arduino Pro Mini and the motors
2. Power lines set to 0.254mm (default width) are too thin to deliver power properly and the power lines should be similar in length
3. The motor driver and the microcontroller are placed on the top layer preventing any copper layer separation between the two components to reduce drone resetting*
4. 90 degree line traces are evident in some of the connections which can cause impedence issues



5. Props are placed incorrectly - still felt air pushed upwards
6. Motor placement is not consistent - motors needs to be at the same start rotation

7. Need to use the exact same components as Max Imagination's design such as 1N4148
8. Need to use a better power switch (SPST) - might not be driving enough current to the motors
9. Filtering capacitors might be needed at the motors to reduce noise
10. PWM signals at the motor gates needs to be sufficient and consistent across the motors
11. Manual testing of the controller linear increase in throttle to check if all motors respond accordingly
12. Individual motors need to be tested at full power with scale load

5.8 Prototype 2.1 - April 26, 2026



This prototype was finally able to lift off, but the flight was still unstable. The PCB re-design is based on the placement of the motor driver circuit at the bottom layer of the PCB along the arms of the drone to provide copper layer separation between the motor driver and the microcontroller to reduce the EMF effects. This design choice was proven to be effective since the drone was more stable and the resetting issue was less frequent which could indicate the EMF effects towards the microcontroller has reduced. In addition, I also added a 100uF capacitor at the power input of the MPU6050 and after adding this capacitor, I could no longer find any occurences of the drone resetting.

In addition, I also used a more effective battery with a higher discharge rate of 30C, 450mAH, and 3.7V. The power lines on the PCB are also thicker which is set to 0.45-0.50mm to reduce current resistance and higher delivery of power across the components. The power lines also have similar lengths to ensure power is distributed evenly across the motors. Lastly, there are no 90 degree traces in the PCB to avoid potential impedence issues.

Furthermore, in this design, I tried balance the weight by distributing the components evenly across the PCB. For example, the ON/OFF switch and the power pins are placed in opposite ends of the drone; the NRF24L01 is centered and the battery is secured at the bottom and centered on the PCB. The motor placements are also more consistent and the wiring is at the same rotation. The propellers are also placed correctly as the air is now being pushed downwards. Next I checked the frequency of the PWM signals at full power to ensure it is consistent across all motors. Using an oscilloscope, I found that each PWM pins outputs a consistent 5.0MHz signal which is sufficient to drive the motors.

In terms of the component choices, I also reverted to using 1N4148 diodes and SI2300DS n-Mosfets for the motor driver as these are commonly used and meets the requirements of this drone. Next I also noticed one of the n-Mosfets burnt out and had to be replaced. Furthermore, I had to add a 10uF capacitor at the power inputs of the NRF24L01 to maintain the signal being transmitted from the controller to the drone. When a signal was lost, the drone kept throttling causing two of the motors (FL and BL) to burn out. After adding the capacitor at the radio module, the communication became more stable and the drone was able to respond properly. Although I also noticed that the front left motor is unstable - sometimes it doesn't spin. Although I did not find any issues in the Mosfets or the diodes in the driver, I suspect poor motor connections is the source of this issue.

In the MultiWii software, I also sumulated multiple drone orientations and movements to ensure the IMU correctly

translates the drone movements. The biggest change was remapping the motor designations in the drone software "output.cpp" based on the following comparisons to various projects and pin designations.

Table 1: Motor Pin Configuration Comparison

Motor	Intended Orientation	Max Imagination (Current Setup)	Electronoobs	Code	Test Results
Motor[0]	BR	D3	D9	D9	FR
Motor[1]	FR	D9	D6	D6	FL
Motor[2]	BL	D5	D5	D5	BL
Motor[3]	FL	D6	D3	D3	BR

It seems that the drone software currently matches Electronoobs' setupe after spinning one motor at a time to see which orientation it maps to. Since I have followed Max Imaginations' PWM wirings, I had to modify the software to reflect this wiring.

- Motor[0] => FR
- Motor[1] => FL
- Motor[2] => BL
- Motor[3] => BR

So the following lines in "output.cpp" was modified to:

Before

```
motor[0] = PIDMIX(-1,+1,-1); //REAR_R
motor[1] = PIDMIX(-1,-1,+1); //FRONT_R
motor[2] = PIDMIX(+1,+1,+1); //REAR_L
motor[3] = PIDMIX(+1,-1,-1); //FRONT_L
```

After

```
motor[0] = PIDMIX(-1,-1,+1); //FRONT_R
motor[1] = PIDMIX(+1,-1,-1); //FRONT_L
motor[2] = PIDMIX(+1,+1,+1); //REAR_L
motor[3] = PIDMIX(-1,+1,-1); //REAR_R
```

Regarding the controller PCB, the transmitted signal is quite unstable. I am suspecting the issue stems from the copper layer is surrounding the pins of the radio module which could cause noisy signals from the transmitter. So I am currently using the controller from the previous prototypes. Otherwise, perhaps filtering capacitors are needed at the power inputs of the controller.

In summary, here are the design choices that made drastic improvements.

- revert the components to 1N4148 Diode, S12300DS n-Mosfet, 10KOhm 0805 resistor
- place a copper layer between the motor drivers and the Atmega328P/MPU6050 for EMF shielding
- thicker (0.45-0.50mm) power lines, zero 90 degree traces, similar power lengths
- 10uF capacitor at the NRF24L01 power pins
- 100uF capacitor at the MPU6050 power pins
- 3.7V 30C 450mAH battery (minimum 25C)
- consistent motor placement - wiring needs to be at the same start rotation

- Minthrottle is set to 1050 based on the comment description “for brushed ESCs like ladybird”
- larger propellers -> larger surface area -> bigger lift

The importance of capacitors at the power pins:

1. No capacitor -> unable to throttle down (motors keeps spinning) - looks like loss of signal at the NRF24L01 module
2. 10uF capacitor at the NRF24L01 power inputs - motors become out of sync - maybe loss of power at the motor terminals
3. 100uF capacitor at the MPU6050 - only the back left motor is not spinning
4. 100uF capacitor at the microcontroller - no change from previous point
5. 470uF capacitor at the battery terminals - front left and back left motors no longer spin

Turns out problems (3-5) are due to the motors burning out from problem 1 which caused overworking in the motors potentially burning inside components.

After these changes, I still face issues with poor motor response at the front left motor. Though it spins sometimes and allows the drone to liftoff, it does lose its spin from time to time. I will try to make these design changes to improve the response.

1. Reinforce connections with external wiring for the front left motor
2. Experiment with various PID values to see if it improves the motor response and stability of the drone
3. Try to add a capacitor at the NRF24L01 power pins of the controller to see if it improves the signal strength and response of the drone

5.9 Prototype 3.0 - September 30, 2026

This prototype mostly involves components changes to the drone and the integration of an FPV camera with a AI detections. This prototype along with the component choices was inspired by DroneScienceTech1993 The PCB shape and layout have been modified to fit the new electrical components of the drone. A PCB thickness was set to 1.0mm to keep it light while maintaining a rigid frame. In terms of component changes, the microcontroller has been modified from an Arduino Pro Mini to an Arduino Nano to support easier debugging with its built in Mini-B USB jack. Furthermore, this board requires an input voltage of 7-12V, so an MT3608 DC-DC boost power converter was integrated to accomodate the new power requirements while maintaining a 3.7V battery to power the motors. A 470uF filtering capacitor was also added at the power input to the motors to filter noise. The n-channel MOSFET at the motor driver was replaced with SI2302DS. A GY-BMP280-3.3 barometric pressure altitude sensor was also added to the drone and enabled inside the drone's firmware. To reduce electrical noise, a 100uF capacitor was attached at the power terminals of the MPU6050 and BMP280 sensors.

In terms of electrical connection changes, I followed Electronoobs setup which connects the motors to the following pin designations of the microcontroller:

- D3 => FL CW,
- D6 => FR CCW
- D5 => BL CCW,
- D9 => BR CW

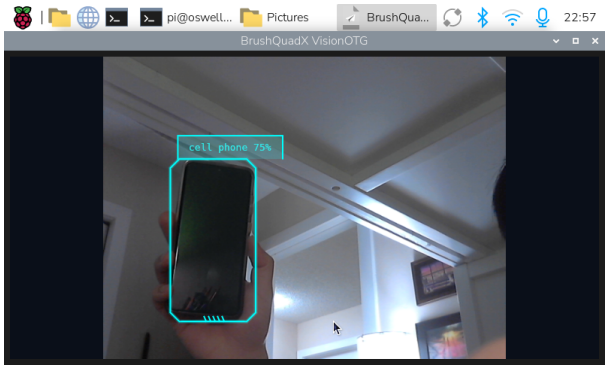
The AUX1 and AUX2 channels are now non-inverted (1000, 2000) in the drone's firmware. The software configuration on "sensors.h" has defined the following aspects

To lift a larger drone with additional components, a larger set of propellers at 46mm with 0.8mm shaft was used to generate a bigger thrust.

In terms of design changes to the controller, the lost of signal could be attributed to the noise generated by the copper layers surrounding the NRF24L01 transmitter on the controller. So the controller PCB was redesigned to add a prohibited copper region around the NRF24L01 to reduce electrical noise.

Here is a list of components used for this prototype: * Si2302 Mosfet n-channel (x4) * 1N148 diode (x4) * 0805 10KOhm resistor (x4) * brush motors (x4) * NRF24L01 tranmitter (x2) * MPU6050 (x1) * BMP280 (x1) * 100uF cap, 470uF cap * MMS1010 D-R slide switch (x1) * 3.7V 30C 450mAH Battery (x1) * MT3608 Boost converter (x1) * Arduino Nano (x2) * 46mm 0.8mm propellers (x4) * MST-12D18G2 SMD Slide Switch - used for the controller (x2) * Joysticks - used for the controller (x2) * AMS1117-3.3 - used for the controller (x1) * SS12SBP2 - used for the controller (x1) * 9V battery - used for the controller (x1) * FPV Reciever 5.8G 150CH OTG USBC (x1) * FPV Camera Micro AIO 600TVL 3.4g 5.8GHz 25mW WT05 (x1)

The biggest feature added to this prototype was the integration of the FPV camera which involves a rust-based software implementation to interface with the drone's camera using an FPV receiver connected either on your Windows PC or on a Raspberry Pi 5. The camera-interface application was written in Rust which picks up the frames received from the camera which is then streamed using a gstreamer pipeline where individual frames can be pulled and run an Ultralytics detection model with embedded NMS to perform object detection on COCO classes.



A video demonstration of the VisionOTG application was uploaded on youtube. Furthermore, installers are available for download on GitHub to support Windows and Raspberry Pi platforms - no extra steps are needed.

5.10 Prototype 3.0 - October 30, 2026

In this log, I have attempted to further stablize the flight of the drone. Here are my attempts.

Looking at the *BrushQuadX_RF24_RXDrone* firmware, the following configuration seems to lift off the drone, but it is very unstable - the drone does not properly lift up straight, but proceeds towards an arbitrary direction. My speculation is that the center of gravity is offset causing the drone to gear towards a different direction.

I have enabled the following definition in the configuration file.

```
#define FORCE_GYRO_ORIENTATION(X, Y, Z) {imu.gyroADC[PITCH] = -Y; imu.gyroADC[ROLL] =  -
↪X; imu.gyroADC[YAW] = -Z;}
```

This definition doesn't seem to correctly translate the IMU movements in MultiWii.

- drone roll = - MultiWii ROLL
- drone pitch = - MultiWii Pitch
- drone yaw = MultiWii yaw

So I made changes to the definition which seems to correct the MultiWii translations

```
#define FORCE_GYRO_ORIENTATION(X, Y, Z) {imu.gyroADC[PITCH] = Y; imu.gyroADC[ROLL] =  X;␣
↪imu.gyroADC[YAW] = -Z;}
```

I tried to lift off the drone, but seems that it just launches the drone forward and damaged the back propeller as it rolled forward. I also tried decreasing P values from the PID for roll and pitch from 3.3 to 2.1, but it did not improve the behaviour. Also the rate and expo for roll and pitch is set to 0.90 and 0.65.

Then I tried the following configuration which is the original source code provided by Electronoobs, but it doesn't properly liftoff (drone just spins uncontrollably), even though this code maps correctly

- motor[0] => rear right
- motor[1] => front right
- motor[2] => rear left
- motor[3] => front left

and on MultiWii roll, pitch, and yaw are translated properly as the orientations.

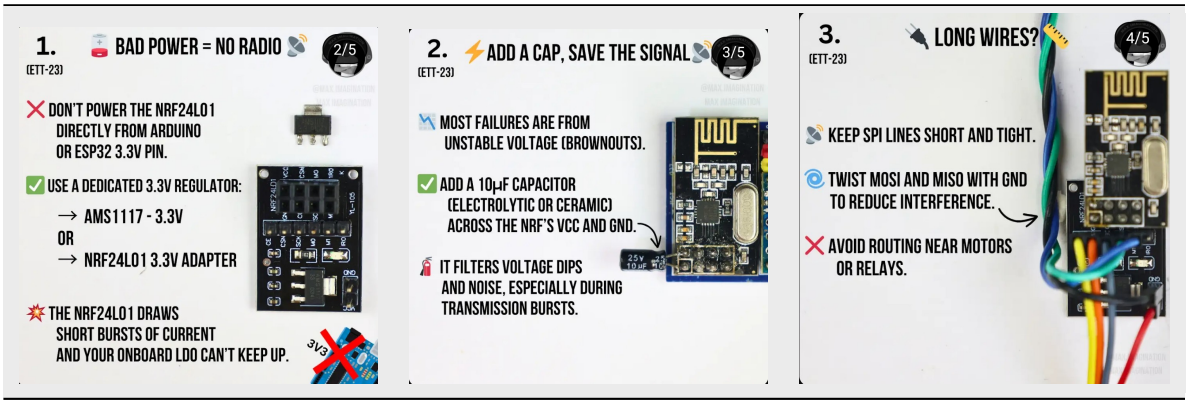
```
motor[0] = PIDMIX(-1,+1,-1); //REAR_R
motor[1] = PIDMIX(-1,-1,+1); //FRONT_R
motor[2] = PIDMIX(+1,+1,+1); //REAR_L
motor[3] = PIDMIX(+1,-1,-1); //FRONT_L
```

Seems that adjusting the battery so that the center of gravity is balanced somewhat improved the drone liftoff, but the due to the excess weight of the components, the drone could not maintain a stable prolonged flight.

5.11 Key Learnings - October 08, 2026

Design Considerations:

1. Copper EMF blocking shield or copper layer separating the motor driver from the MCU helps reduce the EMF effects on the MCU
2. Better to place MPU6050 away from electrical noise of the Arduino and other electrical components
3. The IMU needs to be sitting flat and secured properly to prevent wobbling
4. Capacitors helps reduce power fluctuations and noise
5. Position all 4 motors evenly and in the same manner. The start rotation should be consistent
6. Smaller propeller surface area can cause instability? I think it is better for longer arms and larger propellers to increase the drone surface area - more thrust
7. Thicker drone arms that doesn't bend is better for stability. The overall surface area of the arms and propeller should be maximized
8. Choose your components based on the design specifications
9. Use an oscilloscope to check the PWM signals from the motor gates to see if it is causing sync issues with the motors - ensure consistent PWM frequency across all four motors and enough frequency to drive the motors
10. Test n-channel mosfet Integrity and ensure drain voltage approaches zero as gate voltage increases
11. Here are some tips for using the nRF24L01+ module provided by Max Imagination.



Practical Considerations:

1. Check NRF24 radio communication first before assembling the drone. Always test parts individually
2. Keep it simple and complexity should not come first. Achieve minimum functionality for initial developments
3. Better to use solder with lead, but store properly to prevent contamination (ensure proper storage)
4. Thin kapton tapes do not provide as much insulation as actual electrical tape. Though electrical tape is heavier
5. Ensure the motor power can carry the total load of the drone
6. Ensure prototype works before designing a final PCB

CHAPTER

SIX

FUTURE WORK

1. **Integrate IMU readings towards VisionOTG to see the orientation and tilt of the drone**
    - Separate thread for capturing and displaying drone orientation
    - Integrate roll, pitch, yaw + horizon telemetry visualizations
    - treat drone NRF24L01 as a duplex by receiving controller commands and transmitting IMU readings
2. **Improve model inference time since ONNX model has high latency**
    - optimize the model through quantization and use *ncnn* rust package for running inference
    - Utilize model accelerators for the Raspberry Pi
3. Integrate model segmentation for segmenting detected objects
4. Oswell Avatar/AI integration / drawing
5. **Design ESP32-based brush drone schematic and implement systemctl services**
    - systemctl status visionOTG
    - systemctl status droneControl -> adds a service for controlling the drone
    - Raspberry Pi 5 interface for the controller
    - Custom PCB for Raspberry Pi 5 Joystick + power connections + LCD for display
    - NRF24L01 readings published towards VisionOTG Zenohd topics